Concepts

Slide 5: Process and Multi Threads.

Today's class:

Concepts (Lecture)	• What is a process? Thread? • In Android?
Problem (code)	• What is the limitation of the main thread? • Creating the problem scenario.
Back-to-Concepts (lecture)	• Running on UI thread. • Async Task
Solution (code: all)	• Create the problem scenario • Use Async Task

Program vs. Process

A **program** is a set of instructions + data stored in an executable image.

A **process** is a program “in action”

- Program counter
- CPU registers
- Stacks
- States

For more details:

http://www.tldp.org/LDP/tlk/kernel/processes.html

Program vs. Process

Tea Making Recipe:

1. *Add water in pan.*
2. *Add sugar, tea leaves, spices.*
3. *Bring to boil and simmer.*
4. *Add milk.*
5. *Bring to boil and simmer.*
6. *Strain tea in teapot.*

Program or Process?

Program vs. Process

Process:

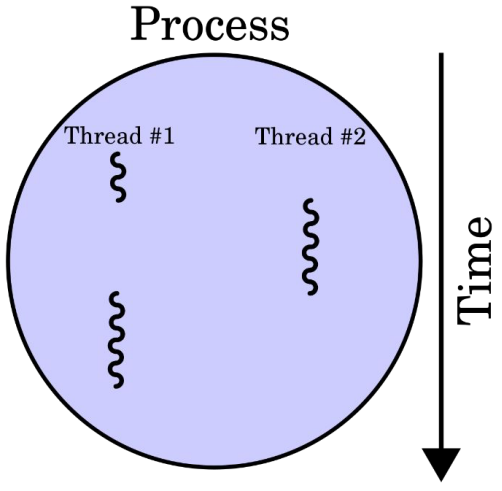
1. *Add water in pan.*
2. *Add sugar, tea leaves, spices.*
3. *Bring to boil and simmer.*
4. *Add milk.*
5. *Bring to boil and simmer.*
6. *Strain tea in teapot.*

Process:

1. *Add water in pan.*
2. *Add sugar, tea leaves, spices.*
3. *Bring to boil and simmer.*
4. *Add milk.*
5. *Bring to boil and simmer.*
6. *Strain tea in teapot.*

Thread

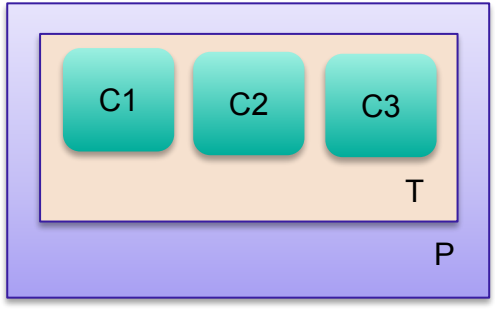
- A **Thread** is a flow of execution inside a process.
- Threads in a process share the same **virtual memory** address space.



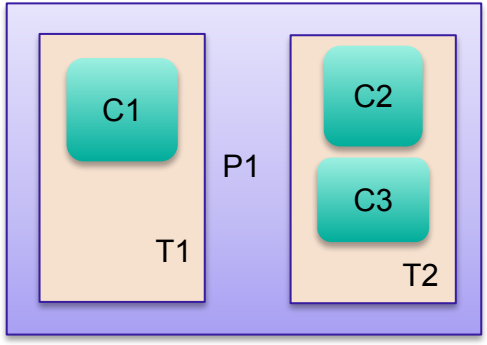
Android Processes and Threads

Default Behavior:

- All components of an App run in the same thread (the main/UI thread) and the same process.



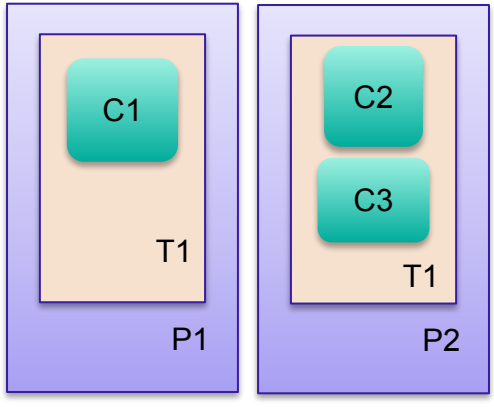
Default



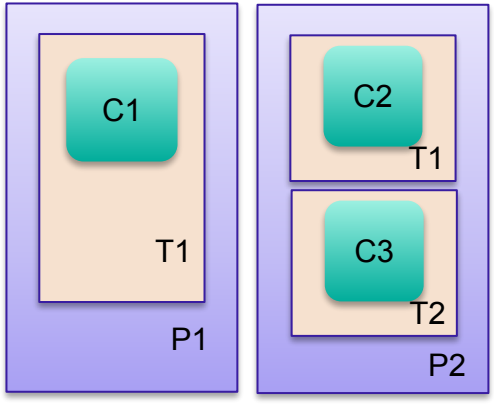
Multiple threads in same process

However, you can:

- Arrange for components to run in different processes.
- Create multiple threads per process.



Multiple processes with single thread in each



Multiple processes with multi threads

Process creation and removal

- **Creation:** When the first component of an App is run and there is currently no process for that App.

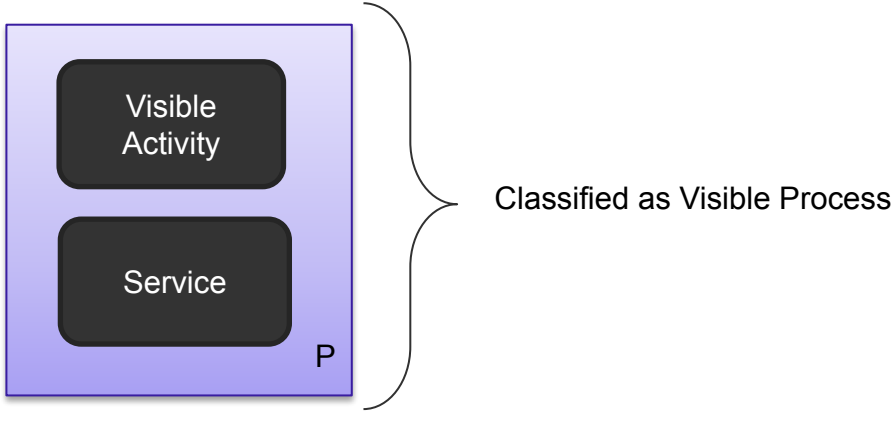
- **Removal:** Remove old processes to reclaim memory for new or more important processes.

Which process should be killed?

Process Hierarchy

- From High to Low Priority

Priority	Type	Description
1	Foreground	On-going interactions; activity, service, broadcast
2	Visible	Not in the foreground, but visible
3	Service	Playing music, downloading stuffs
4	Background	onStop() called, not visible, LRU kill
5	Empty	Lowest priority ← Kill them first



Base Thread

- Act much like usual Java Threads
- Can’t act directly on external User Interface objects
  - Throws the Exception CalledFromWrongThreadException: Only the original thread that created a view hierarchy can touch its views”
- Can’t be stopped by executing destroy() nor stop()
  - Uses instead interrupt() or join() (by case)
- Two main ways of having a Thread execute application code:
  - Providing **a new class that extends Thread and overriding its run()** method
  - Providing **a new Thread instance with a Runnable object** during its creation
  - In both cases, the start() method must be called to actually execute the new Thread.

Base Thread: example1

overriding run()

```
protected void startDownloadThread() {
    Thread t = new Thread() {
        public void run() {
            mResult = "This is new result";
        }
    };
    t.start();
}
```

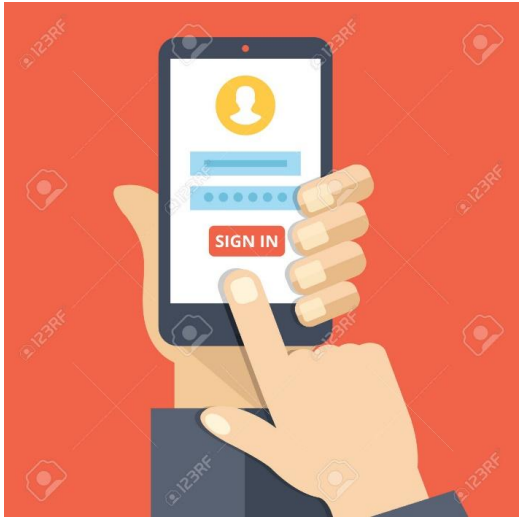
Base Thread: example2

Thread instance with a Runnable

```
class Multi3 implements Runnable{
    public void run(){
        System.out.println("thread is running...");
    }

    public static void main(String args[]){
        Multi3 m1 = new Multi3();
        // Using the constructor Thread(Runnable r)
        Thread t1 = new Thread(m1);
        t1.start();
    }
}
```

Main Thread

- Created when an App is launched
- Often called the **UI thread**



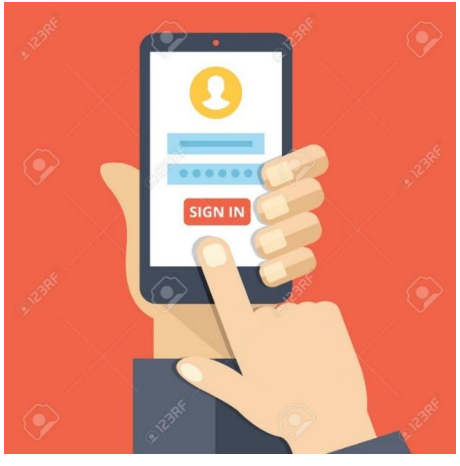
Problem

Keeping Apps Responsive

Problem: keeping an App responsive

- **Testing the limitation of the UI thread:**

- How big a task we **should** do in UI thread?
- How big a task we **can** do in UI thread?
- What is the **alternative**?
- What is the **limitation of this alternative**?



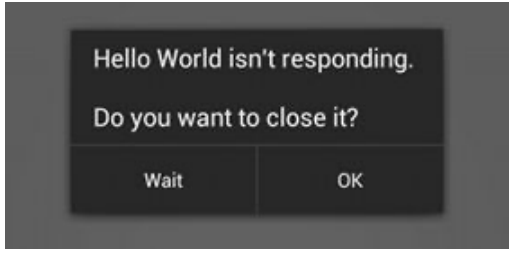

```
void clickEventHandler(View button)
{
     //what is my limit?
     //Let's experiment!
}
```

Worker Thread

- Used to make UI thread light-weight, responsive.
- **Limitation:** Cannot access UI toolkit elements (e.g. views declared in UI thread) from another thread.

So … two lessons to remember

- Do not block the UI thread



Caution: Don't run long running (5 sec+) task in the UI thread

- Do not access the Android UI toolkit from outside the UI thread.

```
public void onClick(View v) {
    new Thread(new Runnable() {
        public void run() {
            Bitmap b = loadImageFromNetwork("http://example.com/i.png");
            mImageView.setImageBitmap(b);
        }
    }).start();
}
```

Solutions

Solution 1. Handler: Consider UI Thread

- A main thread ***Handler*** object
  - Schedules messages and runnable to be executed at some point in the future

Handler Object: Post example

```
public class MainActivity extends AppCompatActivity {
    Handler mHandler;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        mHandler = new Handler();

        new Thread(new Runnable() {
            @Override
            public void run() {
              // perform long running task here
              mHandler.post(new Runnable() {
                  @Override
                  public void run() {
                      mProgressBar.setProgress(currentProgressCount);
                  }
              });
            }
        }).start();
    }
}
```

Handler Object: Post Delayed example

```
public class MainActivity extends AppCompatActivity {
    Handler mHandler;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        mHandler = new Handler();

        new Thread(new Runnable() {
            @Override
            public void run() {
              // perform long running task here
              mHandler.postDelay(new Runnable() {
                  @Override
                  public void run() {
                      mProgressBar.setProgress(currentProgressCount);
                  }
              }, 1000);
            }
        }).start();
    }
}
```

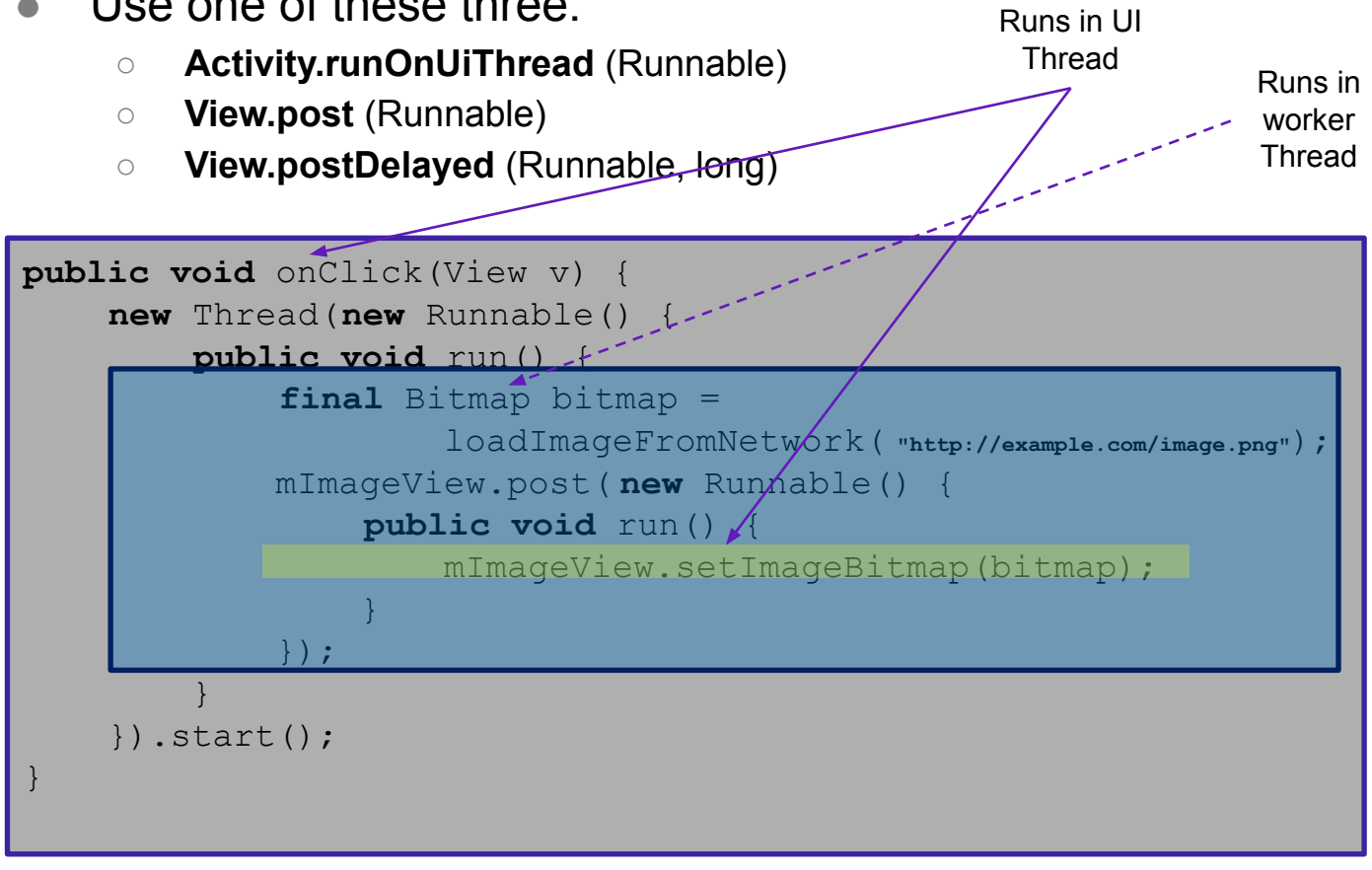
Solution 2. Handler: Consider UI Thread

- posting ***Runnable*** objects to the main view
  - To add an action into a queue performed on a different thread

Posting *Runnable* objects

Access UI Thread from Other Threads

- Use one of these three:
  - **Activity.runOnUiThread** (Runnable)
  - **View.post** (Runnable)
  - **View.postDelayed** (Runnable, long)



Solution 3. Async Task

- Performs the blocking operations in a worker thread, and publishes the results on the UI thread using different built-in methods.

```
public void onClick(View v) {
    new DownloadImageTask().execute( "http://example.com/image.png");
}

private class DownloadImageTask  extends AsyncTask<String, Void, Bitmap> {
    /** The system calls this to perform work in a worker thread and
     * delivers it the parameters given to AsyncTask.execute() */
    protected Bitmap doInBackground(String... urls) {
        return loadImageFromNetwork(urls[0]);
    }

    /** The system calls this to perform work in the UI thread and delivers
     * the result from doInBackground() */
    protected void onPostExecute(Bitmap result) {
        mImageView.setImageBitmap(result);
    }
}
```

Runs in worker Thread

Runs in UI Thread

http://developer.android.com/reference/android/os/AsyncTask.html

AsyncTask

- Created on the UI thread (any worker thread can be created from UI thread)
- Can be executed only once
- Long running tasks run on a background thread and result is published on the UI thread
- Extended as AsyncTask<Void, Void, Void>
    - The three types used by an asynchronous task are the following
        - Params, the type of the parameters sent to the task upon execution
        - Progress, the type of the progress units published during the background computation
        - Result, the type of the result of the background computation

AsyncTask

- Go through 4 steps:
  - **onPreExecute()**: invoked on the UI thread immediately after the execution of the task is started
  - **doInBackground(Param ...)**: invoked on the background thread immediately after onPreExecute() finishes executing
  - **onProgressUpdate(Progress...)**: invoked on the UI thread after a call to publishProgress(Progress...)
  - **onPostExecute(Result)**: invoked on the UI thread after the background computation finishes

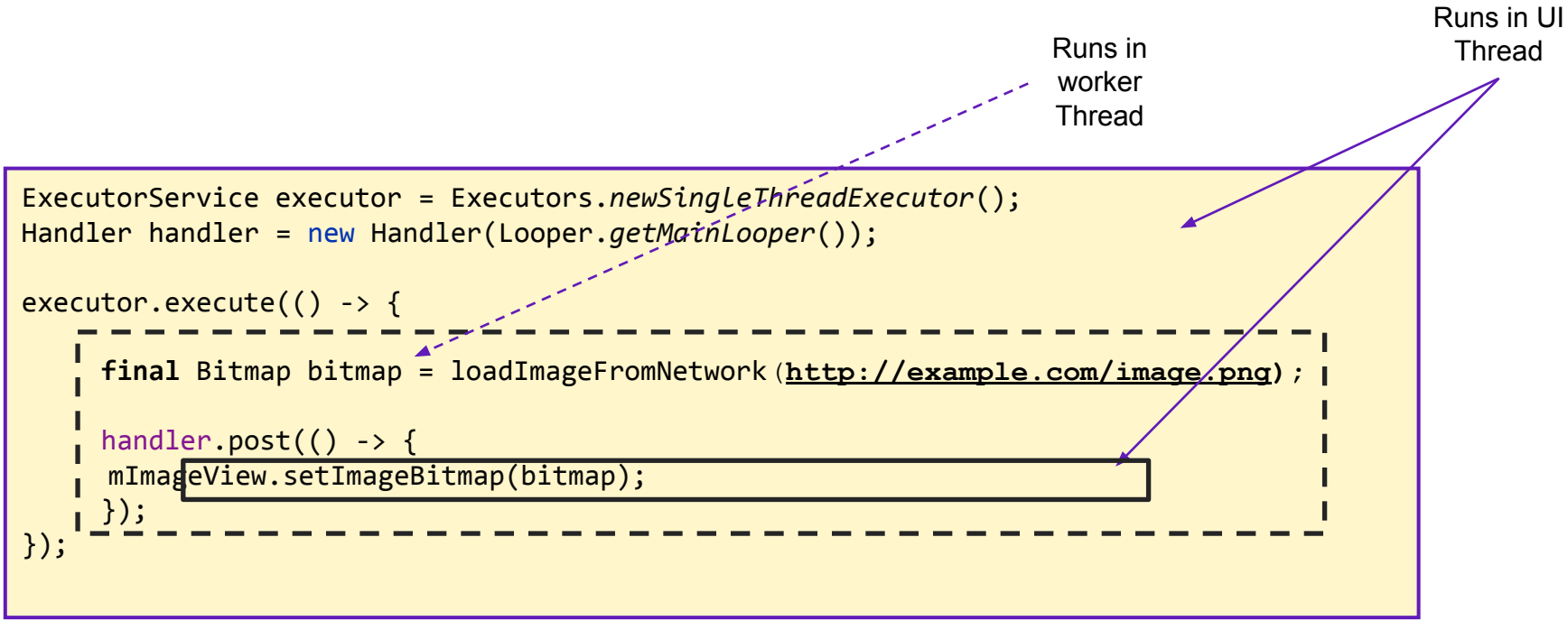
AsyncTask example

```
private class DownloadFilesTask extends AsyncTask<URL, Integer, Long> {
    protected Long doInBackground(URL... urls) {
        int count = urls.length;
        long totalSize = 0;
        for (int i = 0; i < count; i++) {
            totalSize += Downloader.downloadFile(urls[i]);
            publishProgress((int) ((i / (float) count) * 100));
            // Escape early if cancel() is called
            if (isCancelled()) break;
        }
        return totalSize;
    }

    protected void onProgressUpdate(Integer... progress) {
        setProgressPercent(progress[0]);
    }

    protected void onPostExecute(Long result) {
        showDialog("Downloaded " + result + " bytes");
    }
}
```

```
new DownloadFilesTask().execute(url1, url2, url3);
```

Solution 4. Executor and MainLooper (Preferrable)

- Performs the blocking/long running operations in a worker thread (executor), and publishes the results on the UI thread (MainLooper).



```
private void downloadFilesTask(){
    ExecutorService executor = Executors.newSingleThreadExecutor();
    Handler handler = new Handler(Looper.getMainLooper());
    isDownloading = true;
    executor.execute(() -> {
        URL url = new URL(fileUrl);
        urlConnection = (HttpURLConnection) url.openConnection();
        urlConnection.connect();
        InputStream inputStream = urlConnection.getInputStream();
        FileOutputStream fileOutputStream = parent.openFileOutput(fileName, Context.MODE_PRIVATE);
        long totalSize = urlConnection.getContentLength();
        int MEGABYTE = 1024 * 128;
        byte[] buffer = new byte[MEGABYTE];
        long bufferLength;
        long downloaded = 0;
        lastProgress = 0;
        while ((bufferLength = inputStream.read(buffer)) > 0) {
            downloaded += bufferLength;
            int progress = (int) ((downloaded * 100) / totalSize);
            handler.post(() -> {
                selectedTxtProgress.setText(progress + "%");
                selectedProgressBar.setProgress(progress);
            });
            fileOutputStream.write(buffer, 0, (int) bufferLength);
        }
        fileOutputStream.close();
        urlConnection.disconnect();
        handler.post(() -> {
            Toast.makeText(parent, "Download is completed", Toast.LENGTH_LONG).show();
        });
    });
}
```

Different Types of Executor

- **Executors.newCachedThreadPool()** — An ExecutorService with a thread pool that creates new threads as required but reuses previously created threads as they become available.
- **Executors.newFixedThreadPool(int numThreads)** — An ExecutorService that has a thread pool with a fixed number of threads. The numThreads parameter is the maximum number of threads that can be active in the ExecutorService at any one time. If the number of requests submitted to the pool exceeds the pool size, requests are queued until a thread becomes available.
- **Executors.newScheduledThreadPool(int numThreads)** — A ScheduledExecutorServicewith a thread pool that is used to run tasks periodically or after a specified delay.
- **Executors.newSingleThreadExecutor()** — An ExecutorService with a single thread. Tasks submitted will be executed one at a time and in the order submitted.
- **Executors.newSingleThreadScheduledExecutor()** — An ExecutorService that uses a single thread to execute tasks periodically or after a specified delay.

Revisiting Multithreading in Android

```
public void onClick(View v) {
  new Thread(new Runnable() {
    public void run() {
      Bitmap b = loadImageFromNetwork();
      mImageView.setImageBitmap(b);
    }
  }).start();
}
```

- Violate the single thread model: the Android UI toolkit is not thread-safe and must always be manipulated on the UI thread.

- In this piece of code, the ImageView is manipulated on a worker thread, which can cause really weird problems. Tracking down and fixing such bugs can be difficult and time-consuming

```
public void onClick(View v) {
  new Thread(new Runnable() {
    public void run() {
      final Bitmap b = loadImageFromNetwork();
      mImageView.post(new Runnable() {
        public void run() {
          mImageView.setImageBitmap(b);
        }
      });
    }
  }).start();
}
```

- Classes and methods also tend to make the code more complicated and more difficult to read.
- It becomes even worse when our implemented complex operations that require frequent UI updates

```
public void onClick(View v) {
  new DownloadImageTask().execute("http://example.com/image.png");
}

private class DownloadImageTask extends AsyncTask {
     protected Bitmap doInBackground(String... urls) {
         return loadImageFromNetwork(urls[0]);
     }

     protected void onPostExecute(Bitmap result) {
         mImageView.setImageBitmap(result);
     }
 }
```

Cancel Task

- DownloadFilesTask dft = new DownloadFilesTask().execute(url1, url2, url3);
- dft.cancel(true);

Summary

- Executing a long running **blocking task** inside the click event **freezes the UI**.

- Starting a **new thread** to do the blocking task solves the responsiveness problem, but the new thread **cannot access UI elements** declared in UI thread.

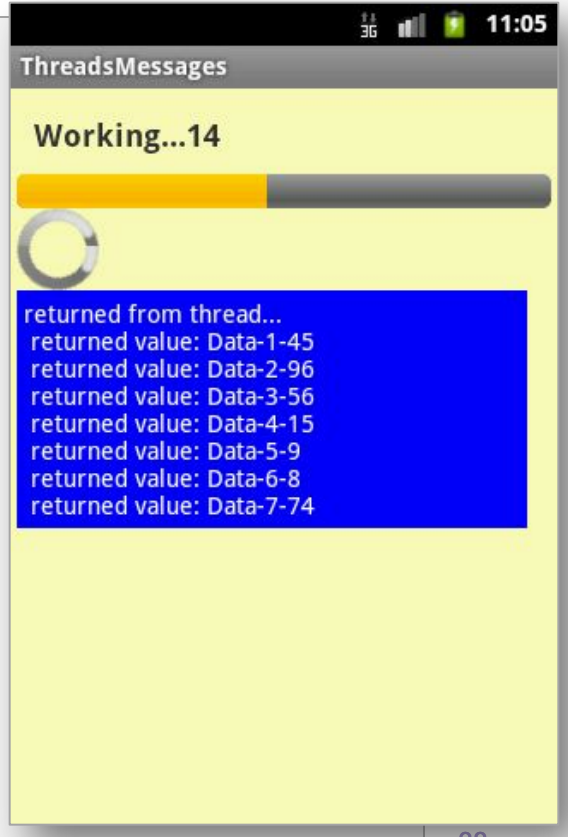
- **Async Task** solves all the problems. It executes **doInBackground()** on a worker thread and **onPostExecute()** on the UI thread.

Example 2. Progress Bar – Using Message Passing

Layout 1/2

The main thread displays a horizontal and a circular *progress bar widget* showing the progress of a slow background operation. Some random data is periodically sent from the background thread and the messages are displayed in the main view.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#44ffff00"
    android:orientation="vertical"
    android:padding="4dp" >

    <TextView
        android:id="@+id/txtWorkProgress"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="10dp"
        android:text="Working ...."
        android:textSize="18sp"
        android:textStyle="bold" />

    <ProgressBar
        android:id="@+id/progress1"
        style="?android:attr/progressBarStyleHorizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <ProgressBar
        android:id="@+id/progress2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
```



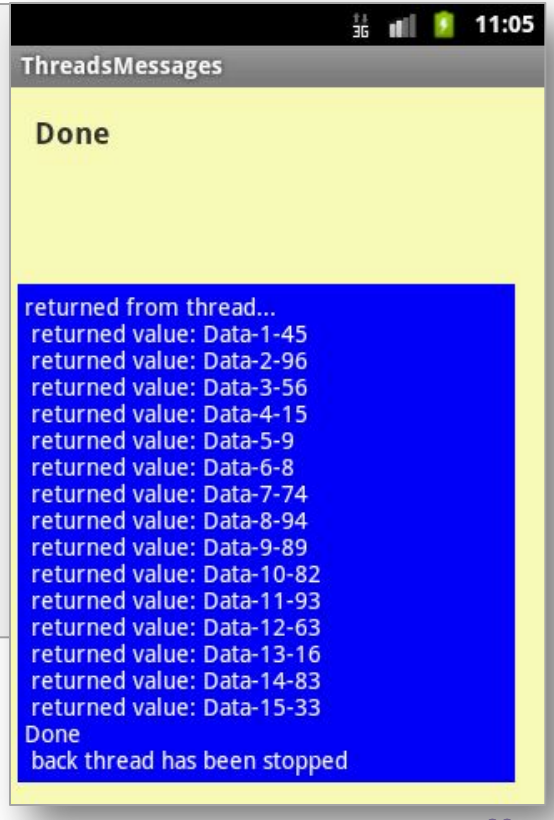
38

Example 2. Progress Bar – Using Message Passing

Layout 2/2

The main thread displays a horizontal and a circular *progress bar widget* showing the progress of a slow background operation. Some random data is periodically sent from the background thread and the messages are displayed in the main view.

```
    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="wrap_content" >

        <TextView
            android:id="@+id/txtReturnedValues"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_margin="7dp"
            android:background="#ff0000ff"
            android:padding="4dp "
            android:text="returned from thread..."
            android:textColor="@android:color/white"
            android:textSize="14sp" />
    </ScrollView>

</LinearLayout>
```

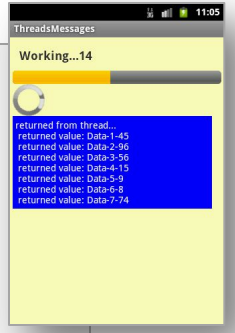


39

Example 2. Progress Bar – Using Message Passing

Activity 1/5

The main thread displays a horizontal and a circular *progress bar widget* showing the progress of a slow background operation. Some random data is periodically sent from the background thread and the messages are displayed in the main view.



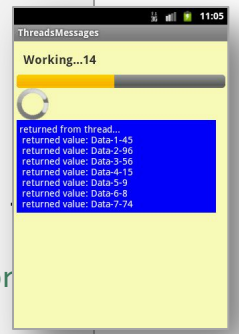

```
public class ThreadsMessages extends Activity {

      ProgressBar bar1;
      ProgressBar bar2;

      TextView msgWorking;
      TextView msgReturned;

      // this is a control var used by backg. threads
      boolean isRunning = false;

      // lifetime (in seconds) for background thread
      final int MAX_SEC = 30;

      //String globalStrTest = "global value seen by all threads ";
      int globalIntTest = 0;
```


40

Example 2. Progress Bar – Using Message Passing

Activity 4/5

```
      public void onStart() {
            super.onStart();
            // this code creates the background activity where busy work is
done
            Thread background = new Thread(new Runnable() {
                  public void run() {
                        try {
                              for (int i = 0; i < MAX_SEC && isRunning; i++) {

                                    //try a Toast method here (it will not wor
                                    //fake busy busy work here
                                    Thread.sleep(1000);  //one second at a time

                                    // this is a locally generated value between
0-100
                                    Random rnd = new Random();
                                    int localData = (int) rnd.nextInt(101);

                                    //we can see and change (global) class
variables
                                    String data = "Data-" + globalIntTest + "-" +
localData;
                                    globalIntTest++;
                                    //request a message token and put some data
in it
                                    Message msg = handler.obtainMessage(1,
(String)data);
```

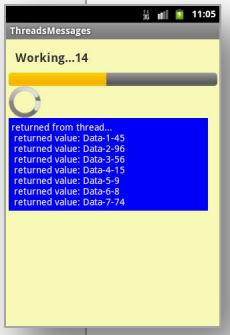


41

Example 2. Progress Bar – Using Message Passing

Activity 5/5

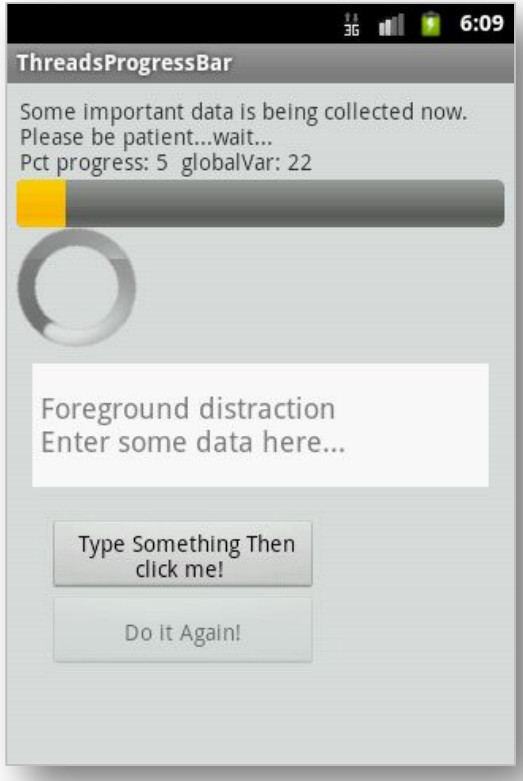
```
 catch (Throwable t) {
                        // just end the background thread
                        isRunning = false;
                        }
                  }
            });// Tread

            isRunning = true;
            background.start();

      }//onStart


      public void onStop() {
            super.onStop();
            isRunning = false;
      }//onStop
}//class
```



42

Example 3. Using Handler post(...) Method

We will try the same problem presented earlier (a slow background task and a responsive foreground UI) this time using the **posting mechanism** to execute foreground ***runnables***.

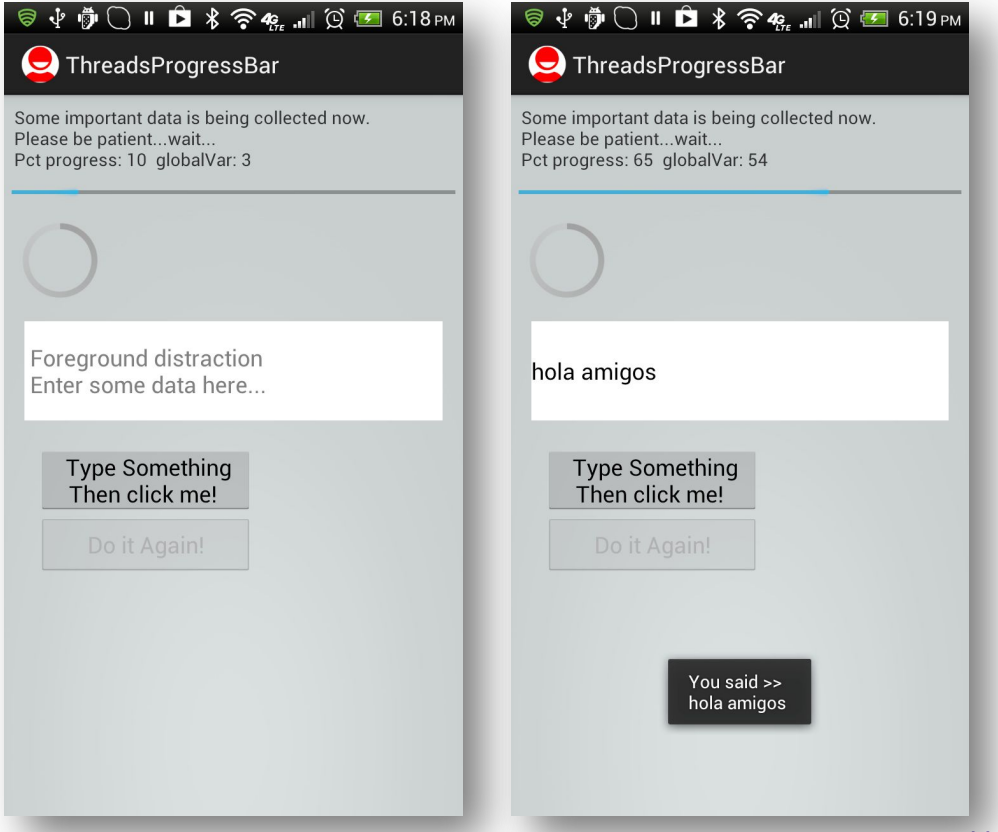


Images obtained from a GingerBread based emulator

43

Example 3. Using Handler post(...) Method

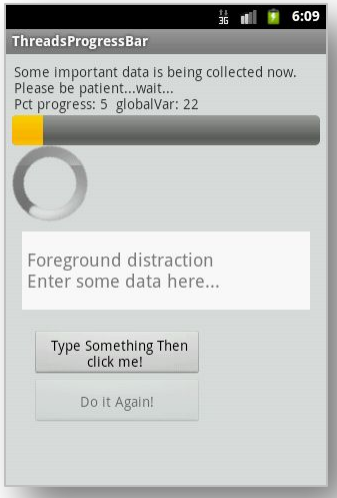
We will try the same problem presented earlier (a slow background task and a responsive foreground UI) this time using the **posting mechanism** to execute foreground ***runnables***.



Images obtained from an IceCream 4.x based device

44

Example 3. Using post - layout: main.xml

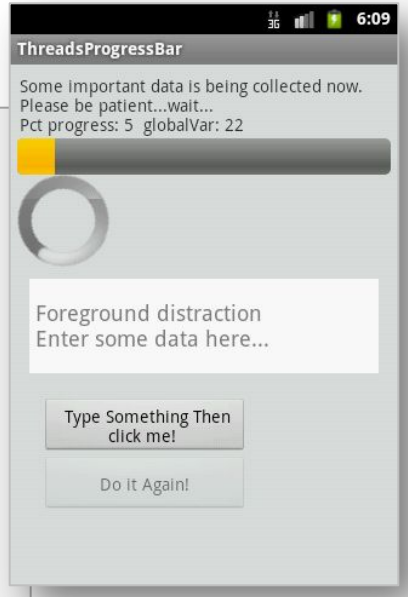



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#22002222"
    android:orientation="vertical"
    android:padding="6dp" >

    <TextView
        android:id="@+id/lblTopCaption"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="2dp"
        android:text=
            "Some important data is been collected now. Patience please..." />

    <ProgressBar
        android:id="@+id/myBarHor"
        style="?android:attr/progressBarStyleHorizontal"
        android:layout_width="match_parent"
        android:layout_height="30dp" />

    <ProgressBar
        android:id="@+id/myBarCir"
        style="?android:attr/progressBarStyleLarge"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
```


45

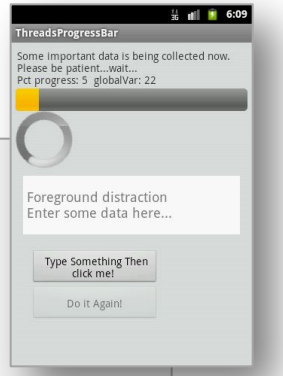
Example 3. Using post - layout: main.xml

```
    <EditText
        android:id="@+id/txtBox1"
        android:layout_width="match_parent"
        android:layout_height="78dp"
        android:layout_margin="10dp"
        android:background="#ffffffff"
        android:textSize="18sp" />

    <Button
        android:id="@+id/btnDoSomething"
        android:layout_width="170dp"
        android:layout_height="wrap_content"
        android:layout_marginLeft="20dp"
        android:layout_marginTop="10dp"
        android:padding="4dp"
        android:text=" Type Something Then click me! " />

    <Button
        android:id="@+id/btnDoItAgain"
        android:layout_width="170dp"
        android:layout_height="wrap_content"
        android:layout_marginLeft="20dp"
        android:padding="4dp"
        android:text=" Do it Again! " />

</LinearLayout>
```



46

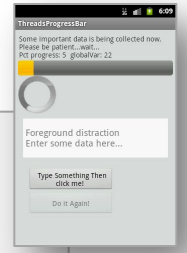
Example 3. Using post - Main Activity




```
public class ThreadsPosting extends Activity {
      ProgressBar myBarHorizontal;
      ProgressBar myBarCircular;

      TextView lblTopCaption;
      EditText txtDataBox;
      Button btnDoSomething;
      Button btnDoItAgain;
      int progressStep = 5;

      int globalVar = 0;
      int accum = 0;

      long startingMills = System.currentTimeMillis();
      boolean isRunning = false;
      String PATIENCE = "Some important data is being collected now. "
                  + "\nPlease be patient...wait... ";

      Handler myHandler = new Handler();

      @Override
      public void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.main);

            lblTopCaption = (TextView) indViewById(R.id.lblTopCaption);
```


47

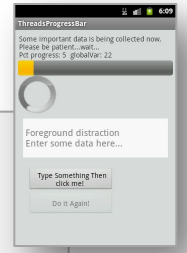
Example 3. Using post - Main Activity




```
            myBarHorizontal = (ProgressBar) findViewById(R.id.myBarHor);
            myBarCircular = (ProgressBar) findViewById(R.id.myBarCir);

            txtDataBox = (EditText) findViewById(R.id.txtBox1);
            txtDataBox.setHint(" Foreground distraction\n Enter some data
here...");

            btnDoItAgain = (Button) findViewById(R.id.btnDoItAgain);
            btnDoItAgain.setOnClickListener(new OnClickListener() {
                  @Override
                  public void onClick(View v) {
                        onStart();
                  }// onClick
            });// setOnClickListener

            btnDoSomething = (Button) findViewById(R.id.btnDoSomething);
            btnDoSomething.setOnClickListener(new OnClickListener() {
                  @Override
                  public void onClick(View v) {
                        Editable text = txtDataBox.getText();
                        Toast.makeText(getBaseContext(), "You said >> \n" + text,
1).show();
                  }// onClick
            });// setOnClickListener

      }// onCreate
```


48

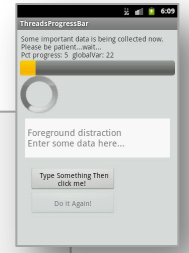
Example 3. Using post - Main Activity




```
@Override
protected void onStart() {
      super.onStart();
      // prepare UI components
      txtDataBox.setText("");
      btnDoItAgain.setEnabled(false);

      // reset and show progress bars
      accum = 0;
      myBarHorizontal.setMax(100);
      myBarHorizontal.setProgress(0);
      myBarHorizontal.setVisibility(View.VISIBLE);
      myBarCircular.setVisibility(View.VISIBLE);

      // create background thread were the busy work will be done
      Thread myBackgroundThread = new Thread( backgroundTask, "backAlias1"
);
      myBackgroundThread.start();

}
```


49

Example 3. Using post - Main Activity



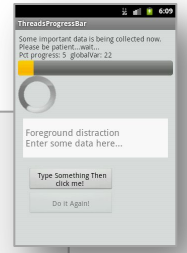

```
// FOREGROUND
// this foreground Runnable works on behave of the background thread
// updating the main UI which is unreachable to it
private Runnable foregroundRunnable = new Runnable() {
      @Override
      public void run() {
            try {
                  // update UI, observe globalVar is changed in back
                  lblTopCaption.setText( PATIENCE
                                    + "\nPct progress: " + accum
                                    + "  globalVar: " + globalVar);

                  // advance ProgressBar
                  myBarHorizontal.incrementProgressBy(progressStep);
                  accum += progressStep;

                  // are we done yet?
                  if (accum >= myBarHorizontal.getMax()) {
                        lblTopCaption.setText("Background work is OVER!");
                        myBarHorizontal.setVisibility(View.INVISIBLE);
                        myBarCircular.setVisibility(View.INVISIBLE);
                        btnDoItAgain.setEnabled(true);
                  }
            } catch (Exception e) {
                  Log.e("<<foregroundTask>>", e.getMessage());
            }
      }
}; // foregroundTask
```

Foreground runnable is defined but not started !

Background thread will requests its execution later

50

Example 3. Using post - Main Activity




```
      // BACKGROUND
      // this is the back runnable that executes the slow work

      private Runnable backgroundTask = new Runnable() {
            @Override
            public void run() {
                  // busy work goes here...
                  try {
                        for (int n = 0; n < 20; n++) {
                              // this simulates 1 sec. of busy activity
                              Thread.sleep(1000);
                              // change a global variable from here...
                              globalVar++;
                              // try: next two UI operations should NOT work
                              // Toast.makeText(getApplication(), "Hi ", 1).show();
                              // txtDataBox.setText("Hi ");

                              // wake up foregroundRunnable delegate to speak for you
                              myHandler.post(foregroundRunnable);
                        }
                  } catch (InterruptedException e) {
                        Log.e("<<foregroundTask>>", e.getMessage());
                  }

            }// run
      };// backgroundTask

}// ThreadsPosting
```

Tell foreground runnable to do something for us…

51

Service in Android

Department of
Computer Science & Engineering

What is Service?

- One of android application components - Activities, **Services**, Content Providers, Broadcast Receivers

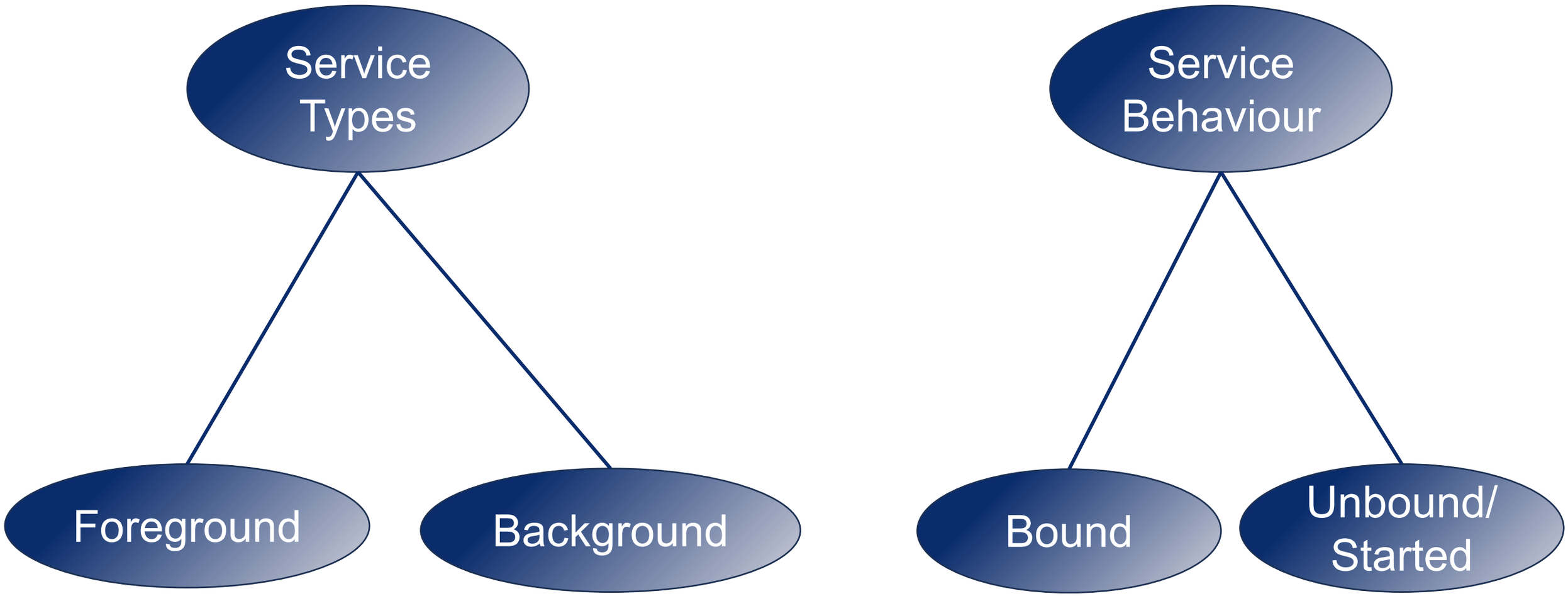
- Usually used to do long-term background work
  - Perform long-running processes without user intervention
  - Have no User Interface

- Can be connected to other components and do inter-process communication (IPC)

- Can run in another process
  - **Service is initiated in UI Thread**

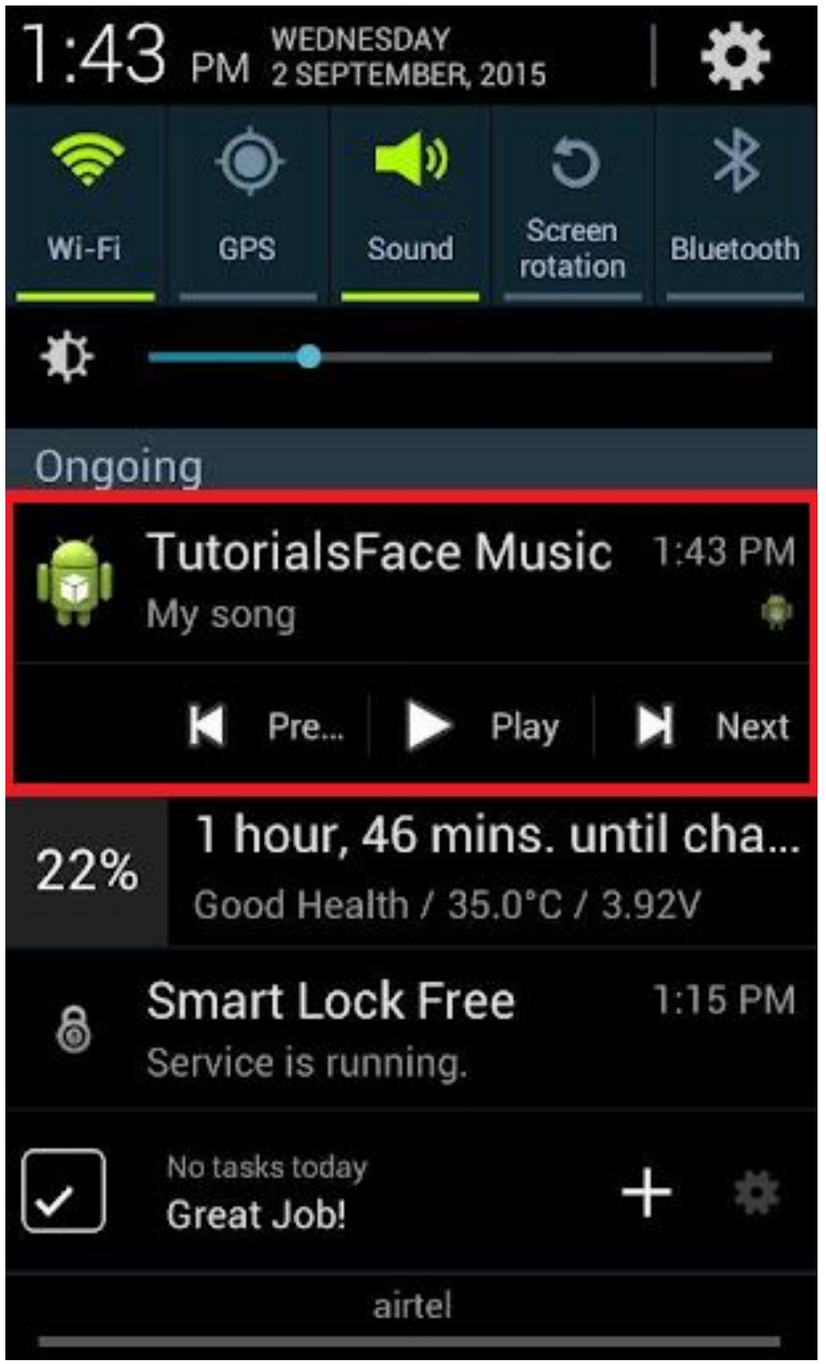
- Need to be declared in AndroidManefest.xml
  - Service can interactive with other component (exported = true)

Android Services



Foreground Services

- Foreground services are those services whose **ongoing tasks** are visible to the users
  - The users can interact with them at ease and track what's happening via **Intent**
- These services continue to run even when users are using other applications
- Examples – Music Player and Downloading

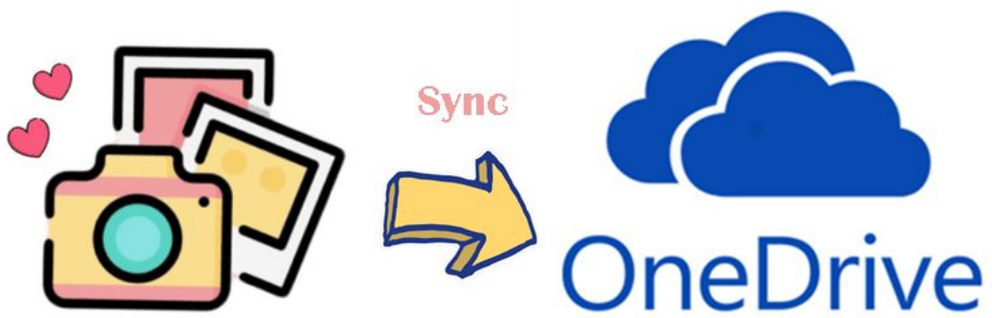


Background Services

- These services run in the background, such that the user can't see or access them

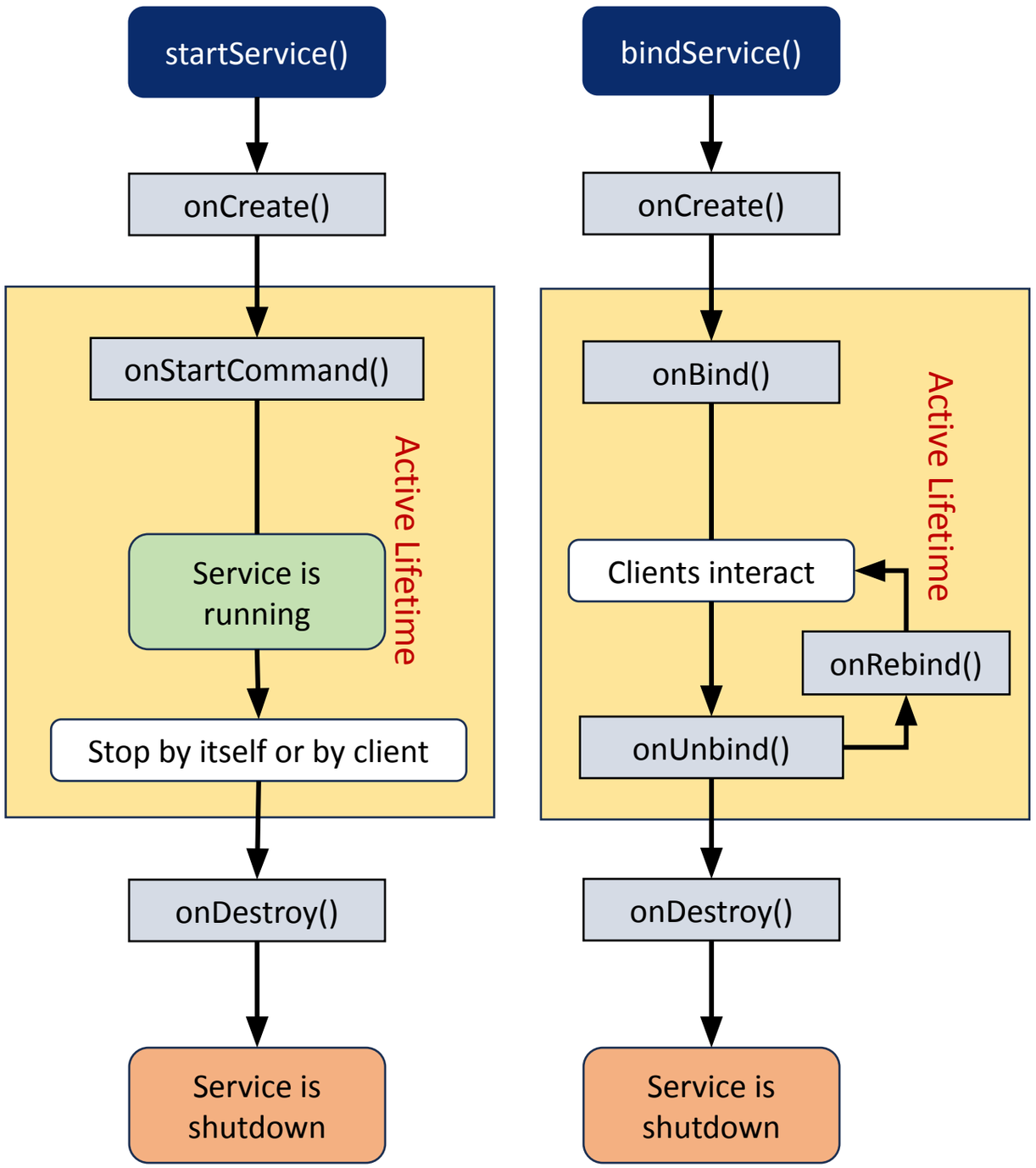
- These are the tasks that don't need the user to know them

- Examples – Syncing and Storing data



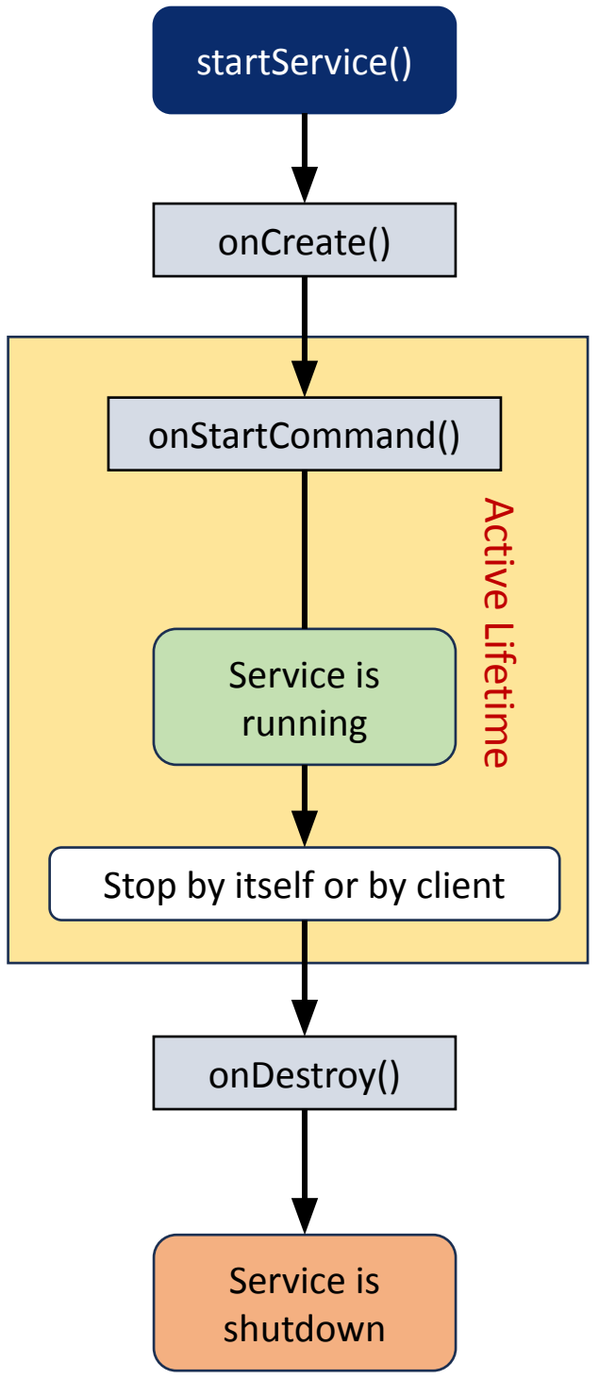
Lifecycle of Android Services

- Android services life-cycle can have two forms of services and they follow two paths, that are:
  - Started Service
    - Also known as Unbound
  - Bound Service

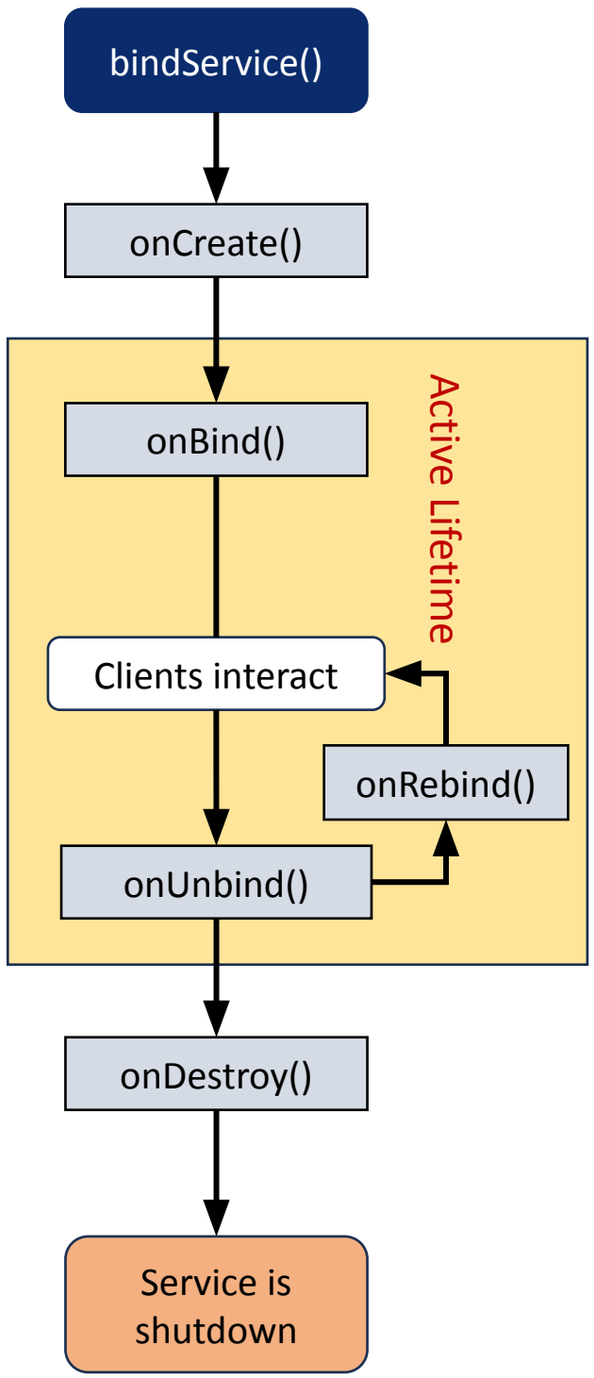


Started/Unbound Service

- Started only when an application component calls startService()
- Performs a single operation and doesn't return any result to the caller
- Once starts, runs in the background even if the component that created it destroys
- Can be stopped only in one of the two cases:
  - By using the stopService() method.
  - By stopping itself using the stopSelf() method



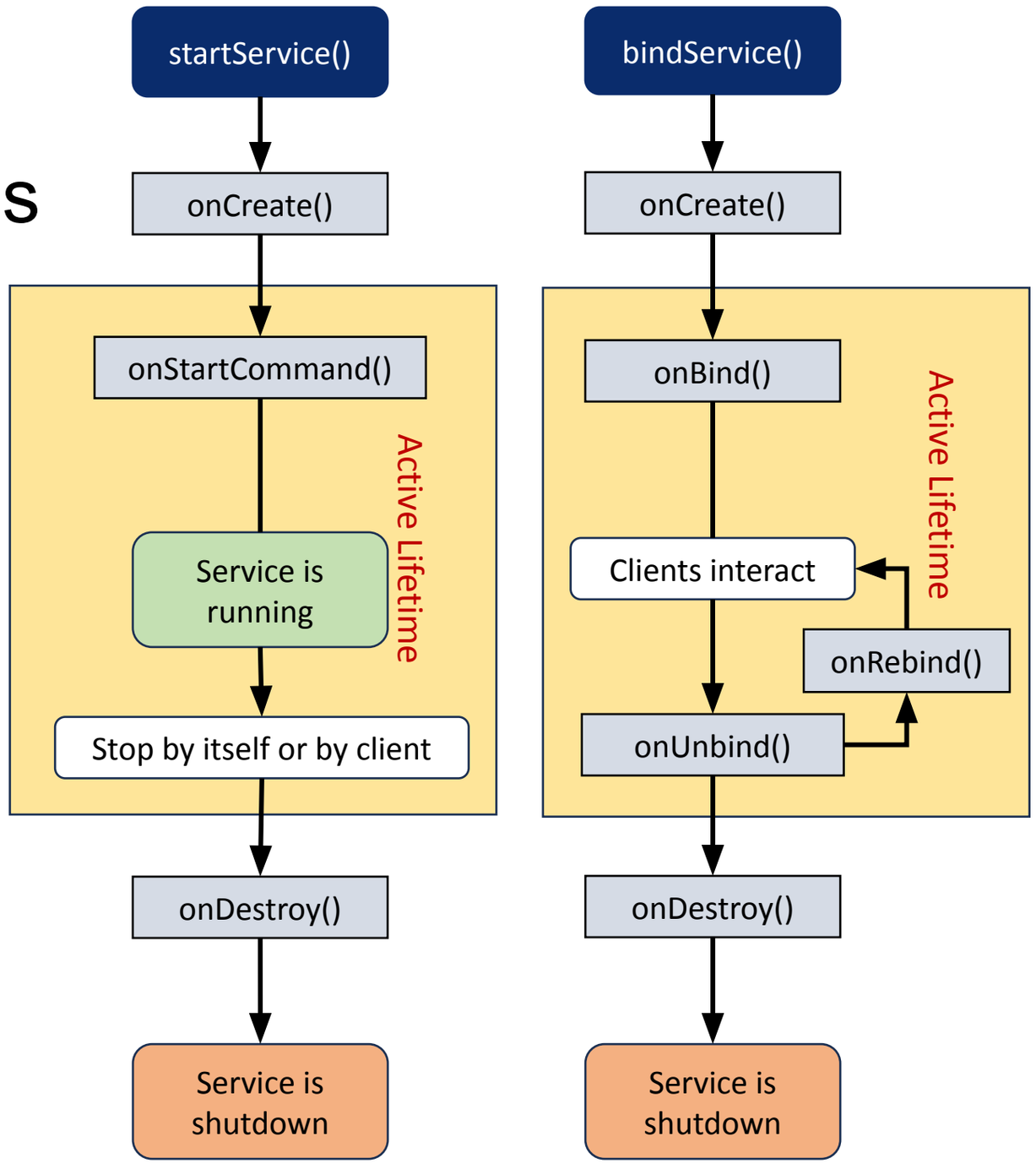
Bound Service



- A service is bound only if an application component binds to it using bindService()
- Gives a client-server relation that lets the components interact with the service
- Components can send requests to services and get results
- Runs in the background as long as another application is bound to it
- Or can be unbound according to our requirement by using the unbindService() method

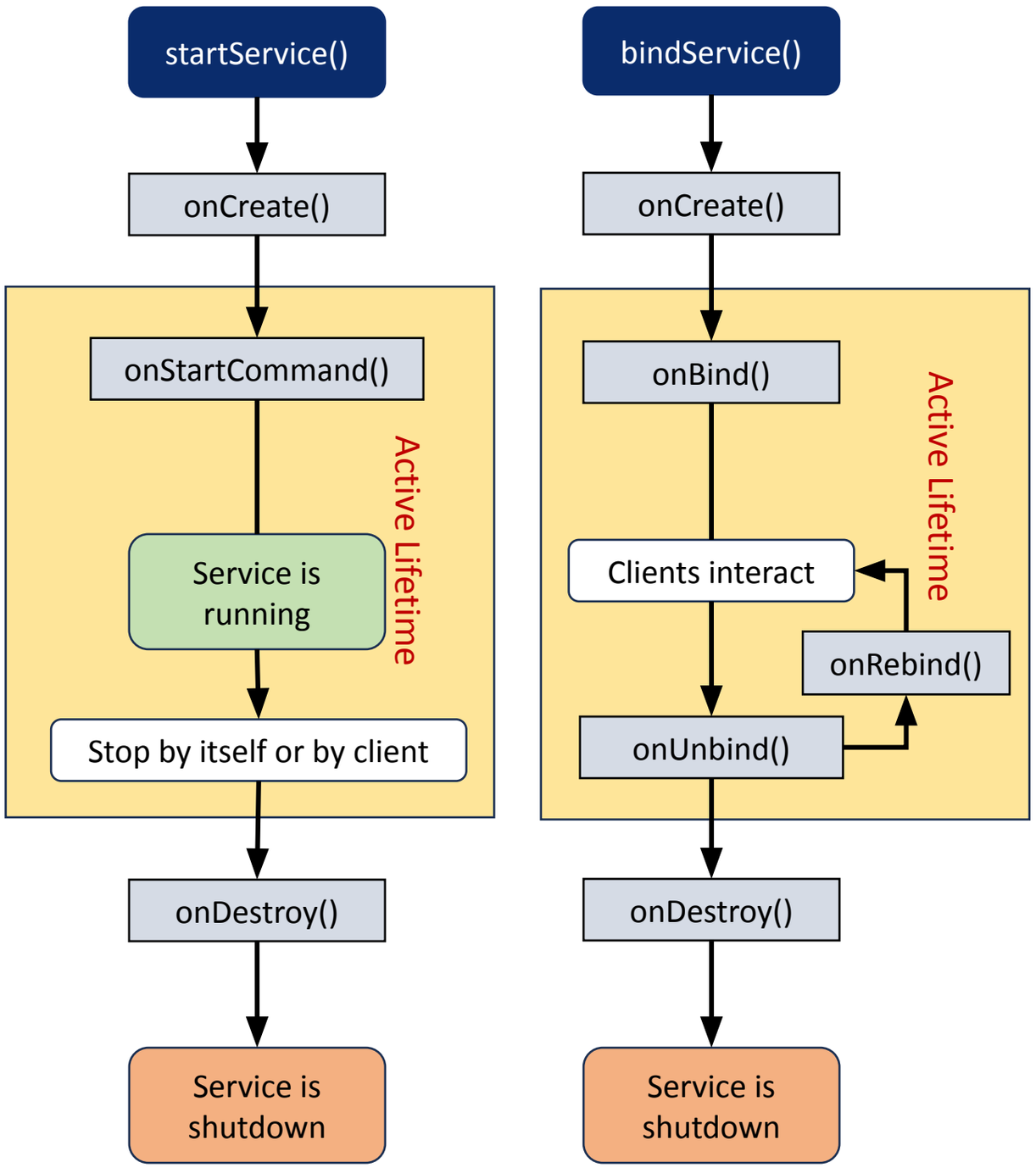
Methods of Android Services

- The service base class defines certain callback methods to perform operations on applications
- The following are a few important methods of Android services :
  - onCreate()
  - onStartCommand()
  - onBind()
  - onUnbind()
  - onRebind()
  - onDestroy()



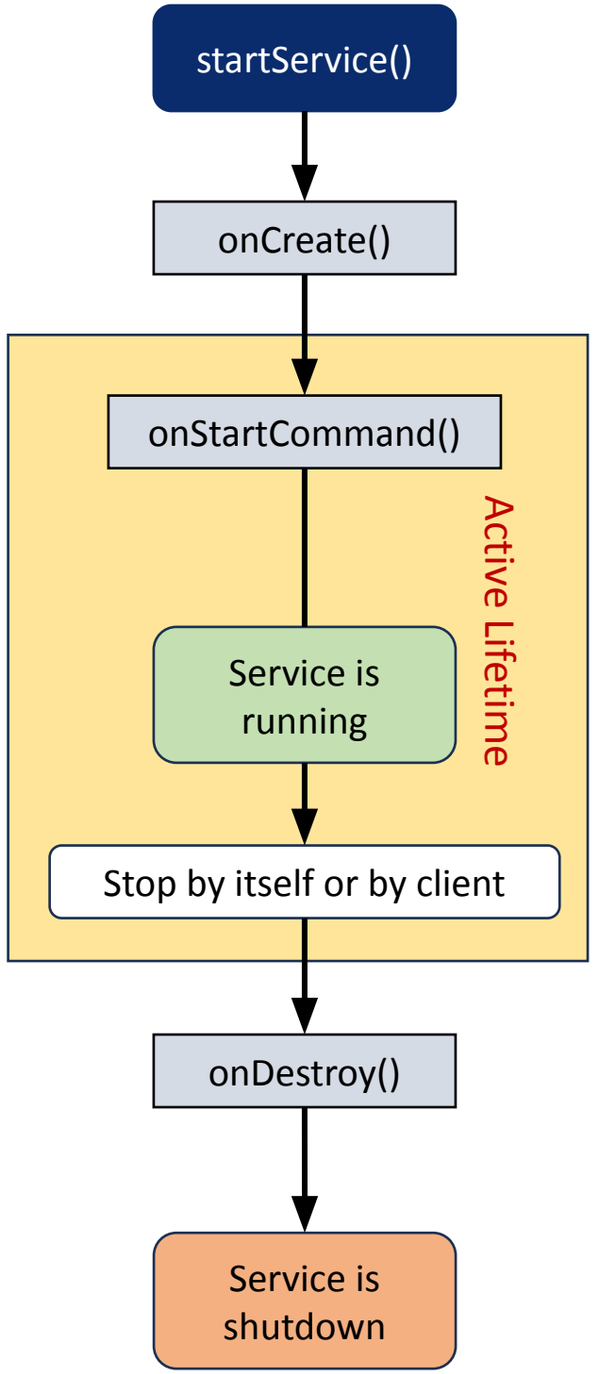
onCreate()

- First method that the system calls when a new component starts the service
- We need this method for a one-time set-up

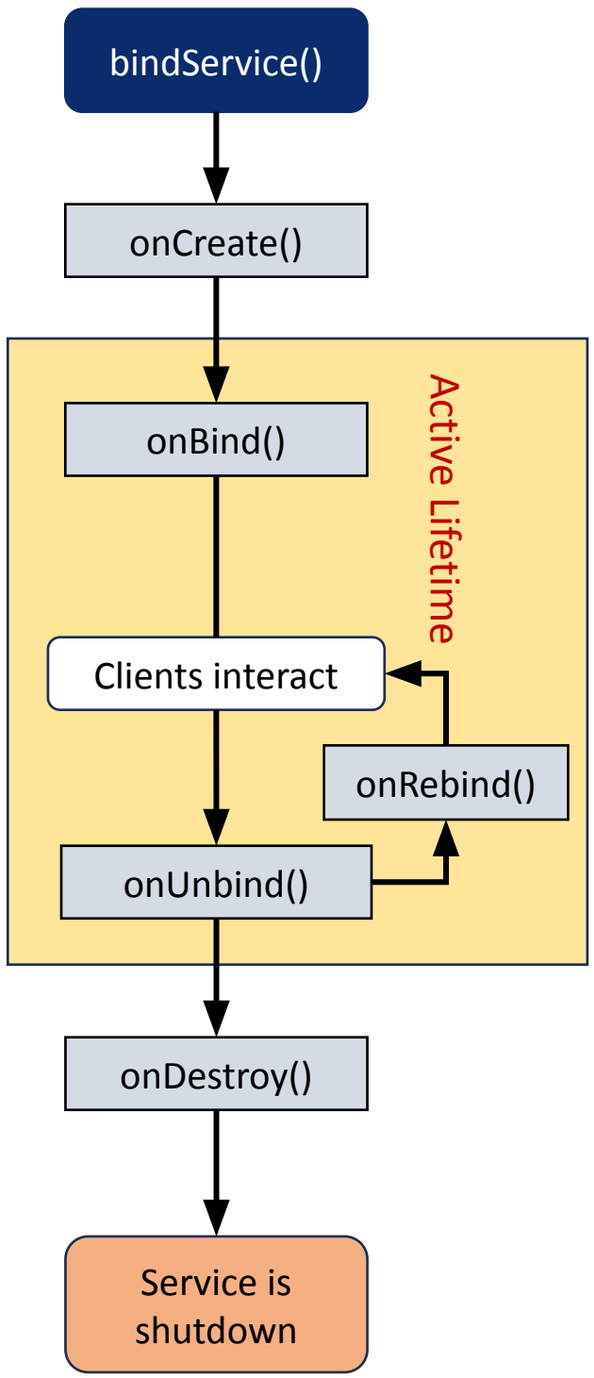


onStartCommand()

- The system calls this method whenever a component, i.e. an activity, requests ‘start’ to a service, using startService()
- Once we use this method it’s our duty to stop the service using stopService() or stopSelf()
- The return value define the behaviour of service
  - return START_STICKY
    - Restart by system if killed
    - Why/How does the system kill a service?
  - return START_NOT_STICKY
    - Doesn’t restart if killed
  - return START_REDELIVER_INTENT
    - Restart if system crashed



onBind()

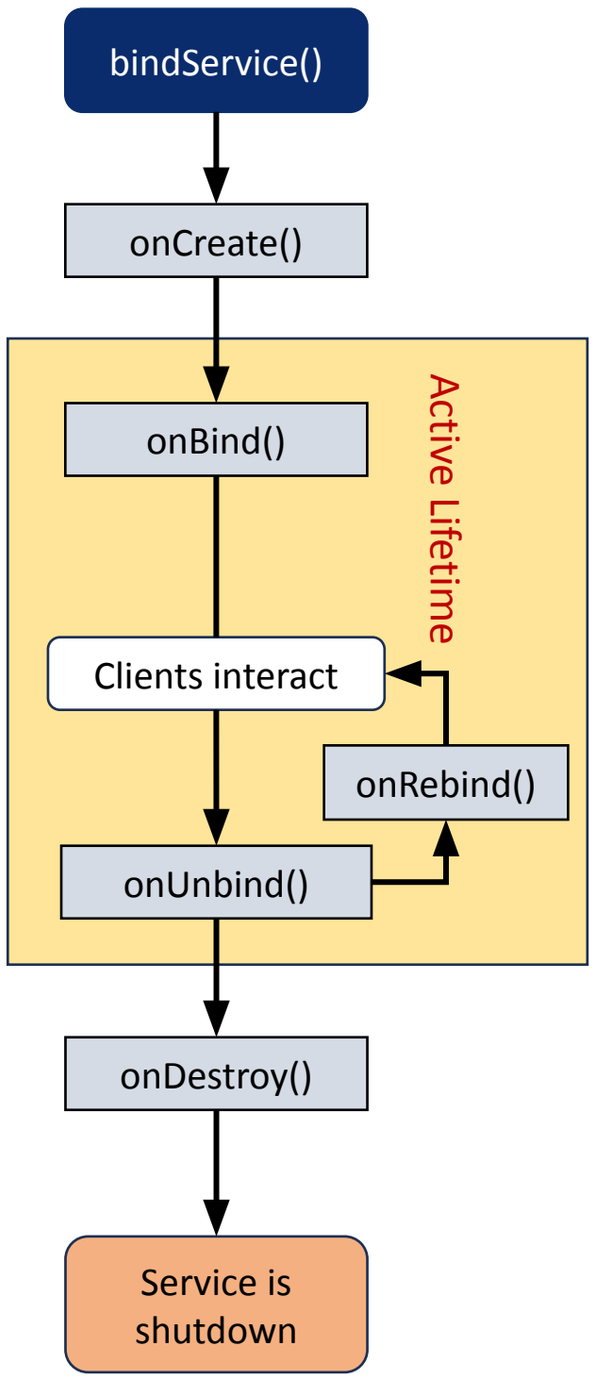


- Invoked when a component wants to bind with the service by calling bindService()

- In this, we must provide an interface for clients to communicate with the service
  - For inter-process communication, we can use the IBinder object

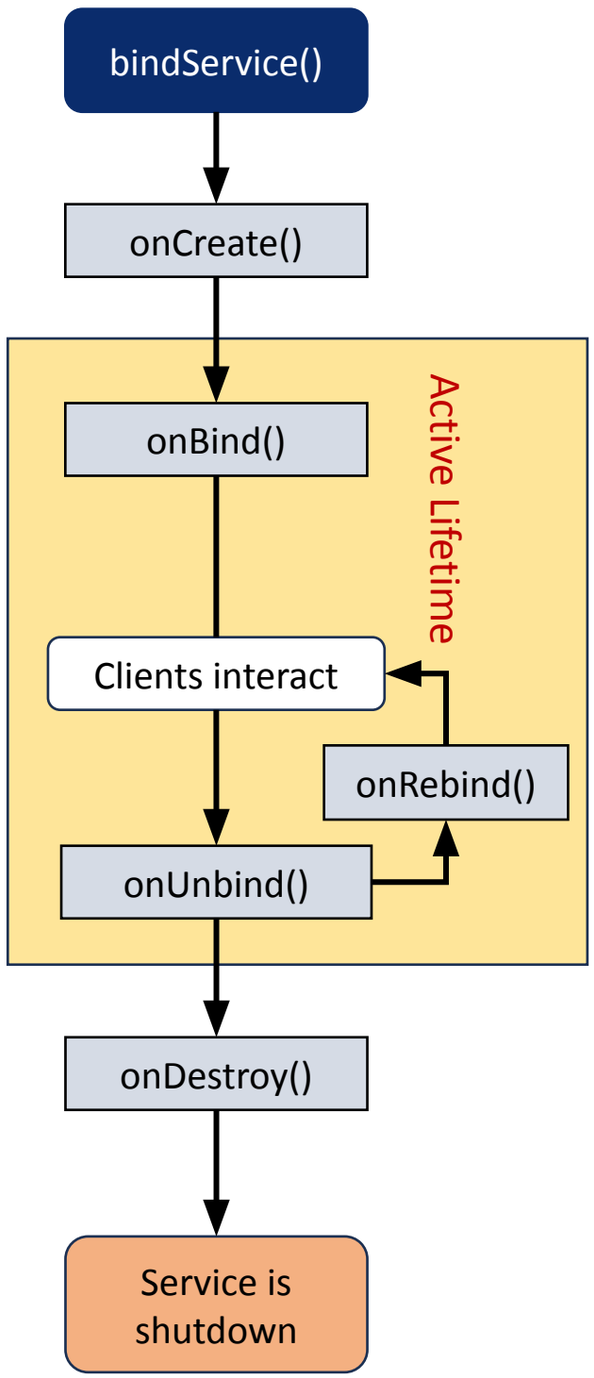
- It is a must to implement this method if bindService() is invoked
  - If in case binding is not required, we should return null as implementation is mandatory.

onUnbind()



- The system invokes this when all the clients disconnect from the interface published by the service

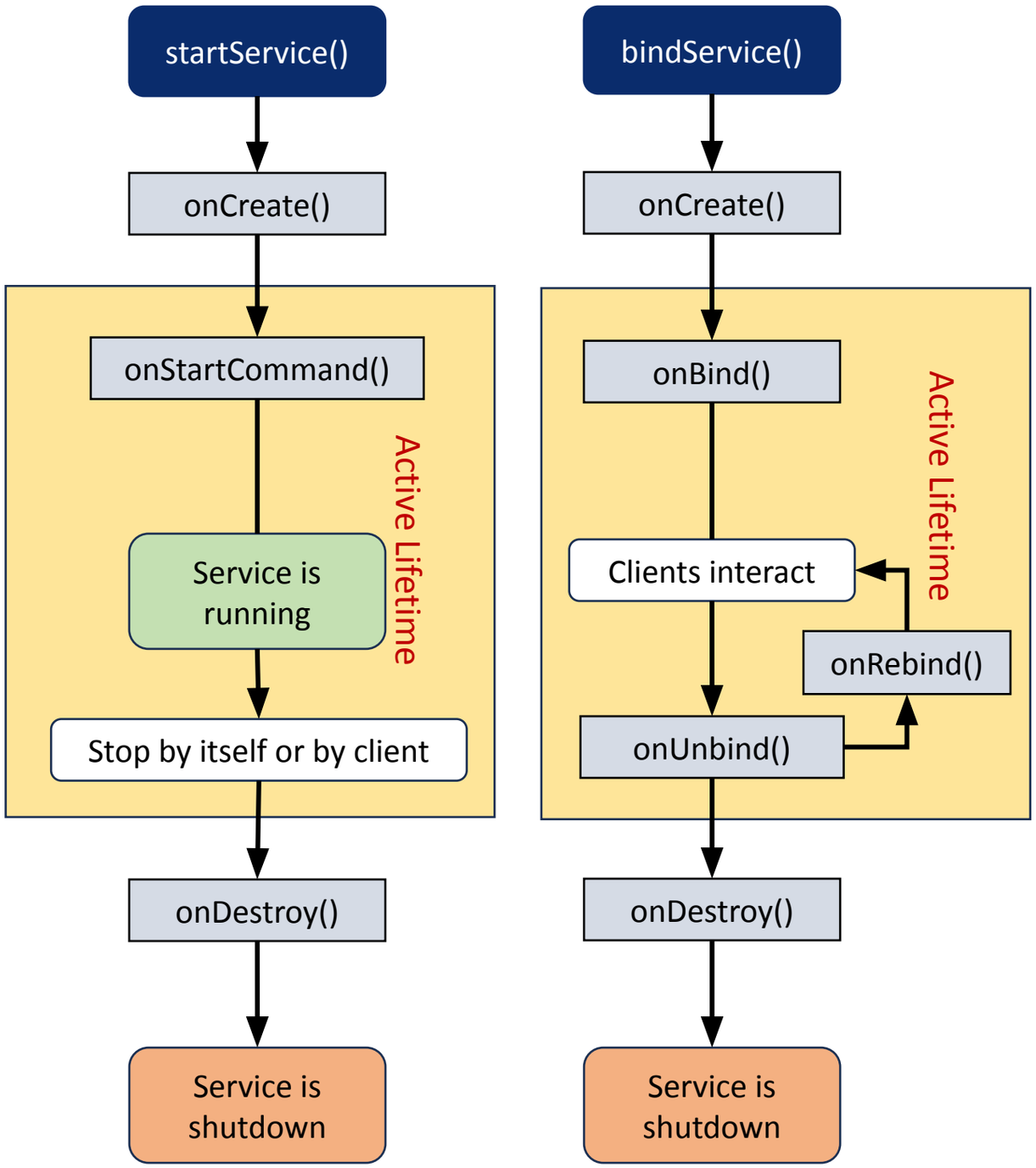
onRebind()



- The system calls this method when new clients connect to the service as long as bound service is running
- The system calls it after the onBind() method

onDestroy()

- The final clean up call for the system
- The system invokes it just before the service destroys
- It cleans up resources like threads, receivers, registered listeners, etc.



Example: Started/Unbound Service

```
public class UnboundService extends Service {
    @Override
    public void onCreare() {
        // one time execution
        super.onCreate();
        // ... here add code for initialization or others
    }
    @Override
    public int onStartCommand(Intent i, int flags, int startId) {
        // This will execute every time when startService() called by the client
        // service is active now
        // ... add code for any processing here
        return START_STICKY; // other choices: START_NOT_STICKY, START_REDELIVER_INTENT
    }
    @Override
    public IBinder onBind(Intent i) {
        // This will never be invoked if startService() called by the client
        throw new UnsupportedOperationException("This service cannot be bound");
    }
    @Override
    public void onDestroy(){
        // ... code to do something before destroying the service process
        super.onDestroy();
    }
}
```

Example: Bound Service

```
public class MyBoundService extends Service {

    // Binder given to clients
    private final IBinder binder = new LocalBinder();

    // Class used for the client Binder
    public class LocalBinder extends Binder {
        public MyBoundService getService() {
            return MyBoundService.this;
        }
    }

    @Override
    public void onCreate() {
        super.onCreate();
        // Called when the service is first created (only
once during its lifetime)
        System.out.println("Service: onCreate called");
    }

    @Override
    public IBinder onBind(Intent intent) {
        // Called when a client (Activity) binds to the
service by calling bindService()
        System.out.println("Service: onBind called");
        return binder;
    }

    // Public method the clients can call
    public String getFromService() {
        return "something from service";
    }
    @Override
    public boolean onUnbind(Intent intent) {
        // Called when all clients have disconnected
from a particular interface published by the service
        // If we return true, onRebind() will be
called when a new client binds
        System.out.println("Service: onUnbind
called");
        return true;
    }

    @Override
    public void onRebind(Intent intent) {
        // Called when a client rebinds to the
service after it had been unbound (and we returned
true in onUnbind)
        System.out.println("Service: onRebind
called");
        super.onRebind(intent);
    }

    @Override
    public void onDestroy() {
        // Called when the service is no longer used
and is being destroyed
        System.out.println("Service: onDestroy
called");
        super.onDestroy();
    } // end of onDestroy()


} // end of service class
```

Example: Bound Service

```
public class MainActivity extends Activity {
    private MyBoundService myService;
    private boolean isBound = false;
    private TextView textView;
    private Button btnBind, btnUnbind, btnGetFromService;

    // Defines callbacks for service binding,
      // passed to bindService()
    private ServiceConnection connection = new
ServiceConnection() {

      public void onServiceConnected(ComponentName name,
IBinder service) {
         // Called when the connection with the service has
been established
         System.out.println("onServiceConnected");
         MyBoundService.LocalBinder binder =
(MyBoundService.LocalBinder) service;

          // Get service instance
         myService = binder.getService();
         isBound = true;
      }

      public void onServiceDisconnected(ComponentName name)
{
        // Called when the connection with the service has
been unexpectedly disconnected – crashed or killed
        System.out.println("onServiceDisconnected");
        isBound = false;
      }
  };
```

```
  protected void onCreate(Bundle savedInstanceState)
{
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);

      // Set up button click listeners
      btnBind.setOnClickListener(v->{
         // Called when user presses "Bind Service"
          // Bind to the service
          Intent intent = new Intent(this,
MyBoundService.class);
     bindService(intent, connection,
Context.BIND_AUTO_CREATE);
      });
      btnUnbind.setOnClickListener(v->{
        // Called when user presses ”Unbind Service"
        unbindMyService();
      });
      btnGetFromService.setOnClickListener(v->{
        // Called when user presses "Get Time"
        if (isBound) {
           String str = myService.getFromService();
           textView.setText(str);
        }
      });
  }

  private void unbindMyService() {
      // Unbind from the service
      if (isBound) {
         unbindService(connection); isBound = false;
      }
  }

  protected void onDestroy() {
    super.onDestroy();
    // Good practice: unbind service to avoid leaks
    unbindMyService();
  }
} // end of activity class
```

	Bound Service	**Started Service**
How you start	bindService(intent, conn, flags)	startService(intent)
Needs ServiceConnection?	✅ Yes	❌ No
Direct method call?	✅ Yes (through the Binder)	❌ No (need other IPC methods)
Service lifespan	Exists only while bound	Lives until stopped manually or system kills it

Singleton Class

```
Intent startIntent = new Intent(this, MyStartService.class);
// ... do intent.putExtra([your data]) if need
startService(startIntent);

Intent stopIntent = new Intent(this, MyStartService.class);
stopService(stopIntent);
```

Refer to same service

Declaration of Service in AndroidManifest

- android:name="[package/service]"
- android:enabled="[true|false]"
- android:exported="[true|false]"
- android:isolatedProcess="[true|false]"
- android:process="[name/of/process]"

```
<service
    android:name=".MyIntentService"
    android:enabled="true"
    android:exported="false" />
<service
    android:name=".MyStartService"
    android:enabled="true"
    android:exported="false" />
<service
    android:name=".MyLocalBindService"
    android:enabled="true"
    android:exported="true" />
<service
    android:name=".MyMessageQueueService"
    android:enabled="true"
    android:exported="true"
    android:isolatedProcess="true"
    android:process="ServiceProcess" />
<service
    android:name=".MyAIDLService"
    android:enabled="true"
    android:exported="true"
    android:isolatedProcess="true"
    android:process="ServiceProcess" />
```

Isolated Service

- android:isolatedProcess=“true”
- android:process=“[process_name]”
- Even you give two isolated processes the same process name, they will NOT run in same process.

Thus, you get at least 3 process: app, service1, and service2

The process names of service1 and service2 are same

Exported Service

- Exported Service makes your app be used from other application's service
- The exported service is run in the process of its application, NOT in the process of caller

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="idv.chatea.servicedemo" >
```

```xml
<service
    android:name=".MyExportedService"
    android:enabled="true"
    android:exported="true">
    <intent-filter>
        <action android:name="ExportedService" />
    </intent-filter>
</service>
```

```java
@Override
protected void onStart() {
    super.onStart();

    Intent intent = new Intent("ExportedService");
    intent.setPackage("idv.chatea.servicedemo");
    startService(intent);
}

@Override
protected void onStop() {
    Intent intent = new Intent("ExportedService");
    intent.setPackage("idv.chatea.servicedemo");
    stopService(intent);

    super.onStop();
}
```

Notification

- Notification is part of Service
- Use NotificationCompat.Builder
- startForeground(int notificationId, Notification)
  # the notificationId must NOT be 0
- startForeground with same id will replace the previous notification which has same id.

Example: Notification

```
private static final int NOTIFICATION_ID = 1;

private void showNotification() {
    NotificationCompat.Builder builder = new NotificationCompat.Builder(this);

    // builder.setXXX ...

    startForeground(NOTIFICATION_ID, builder.build());
}

private void hideNotitification() {
    /** true: remove notification. false: don't remove notification **/
    stopForeground(true);
}
```

Intent Service: an additional service class

- A base class for services to handle asynchronous requests
- Executes long-running programs without affecting any user's interface interaction

- Intent services run and execute in the background

- Certain important features of Intent Service are:
  - Queues up the upcoming request and executes one by one
  - Once the queue is empty it stops itself, without the user's intervention in its lifecycle
  - Does proper thread management by handling the requests on a separate thread

```java
public class MyIntentService extends IntentService {

    private static final String ACTION_FOO = "idv.chatea.servicedemo.action.FOO";
    private static final String ACTION_BAZ = "idv.chatea.servicedemo.action.BAZ";

    private static final String EXTRA_PARAM1 = "idv.chatea.servicedemo.extra.PARAM1";
    private static final String EXTRA_PARAM2 = "idv.chatea.servicedemo.extra.PARAM2";

    public MyIntentService() { super("MyIntentService"); }

    public static void startActionFoo(Context context, String param1, String param2) {
        Intent intent = new Intent(context, MyIntentService.class);
        intent.setAction(ACTION_FOO);
        intent.putExtra(EXTRA_PARAM1, param1);
        intent.putExtra(EXTRA_PARAM2, param2);
        context.startService(intent);
    }

    public static void startActionBaz(Context context, String param1, String param2) {
        Intent intent = new Intent(context, MyIntentService.class);
        intent.setAction(ACTION_BAZ);
        intent.putExtra(EXTRA_PARAM1, param1);
        intent.putExtra(EXTRA_PARAM2, param2);
        context.startService(intent);
    }

    @Override
    protected void onHandleIntent(Intent intent) {
        if (intent != null) {
            final String action = intent.getAction();
            if (ACTION_FOO.equals(action)) {
                final String param1 = intent.getStringExtra(EXTRA_PARAM1);
                final String param2 = intent.getStringExtra(EXTRA_PARAM2);
                // ... do something for Action Foo ...
            } else if (ACTION_BAZ.equals(action)) {
                final String param1 = intent.getStringExtra(EXTRA_PARAM1);
                final String param2 = intent.getStringExtra(EXTRA_PARAM2);
                // ... do something for Action Baz ...
            }
        }
    }
}
```

Broadcast

Customized by Rasel

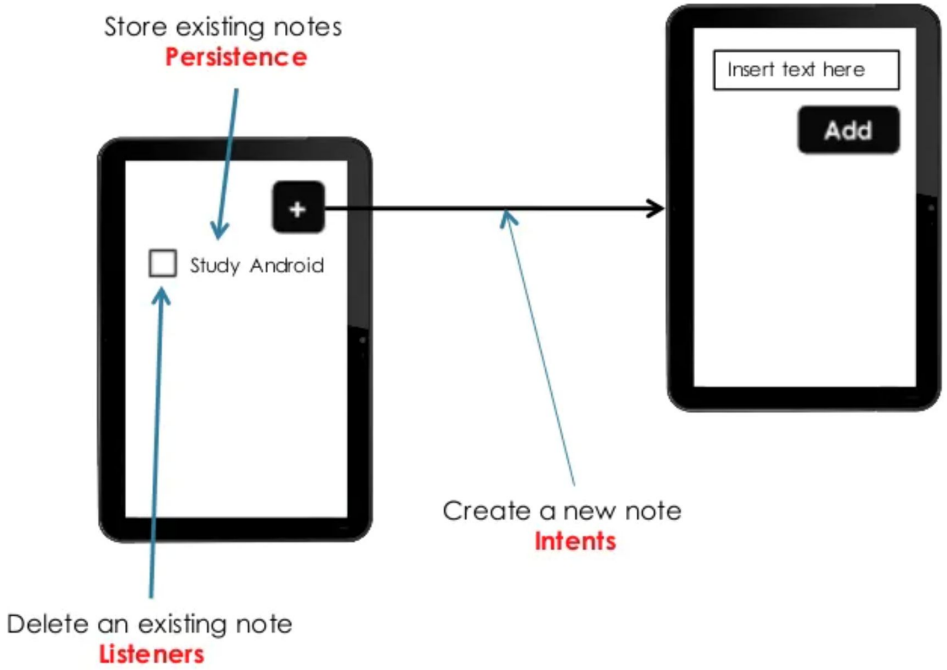
This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Contents

- Broadcast intents
- Broadcast receivers
- Implementing broadcast receivers
- Custom broadcasts
- Security
- Local broadcasts

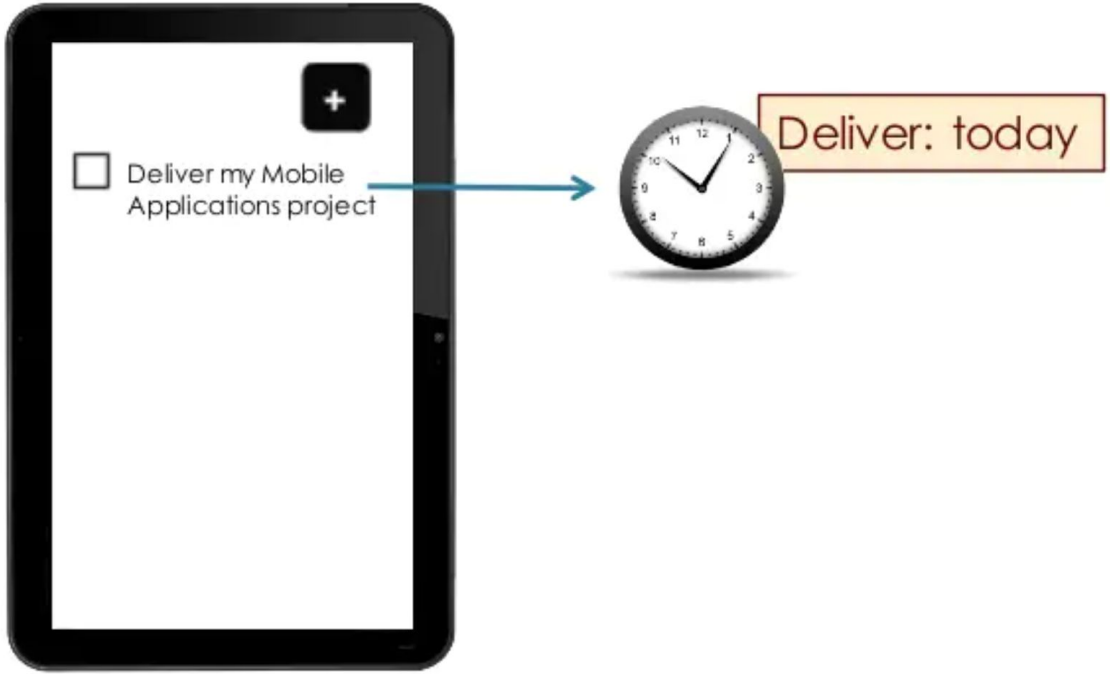
This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

App Feature: Take Note



CC BY NC

App Feature: Remind something in a specific time



Merging both features: Reminder for a task

Creating a new note

The user decides whether to associate an **alarm** with it

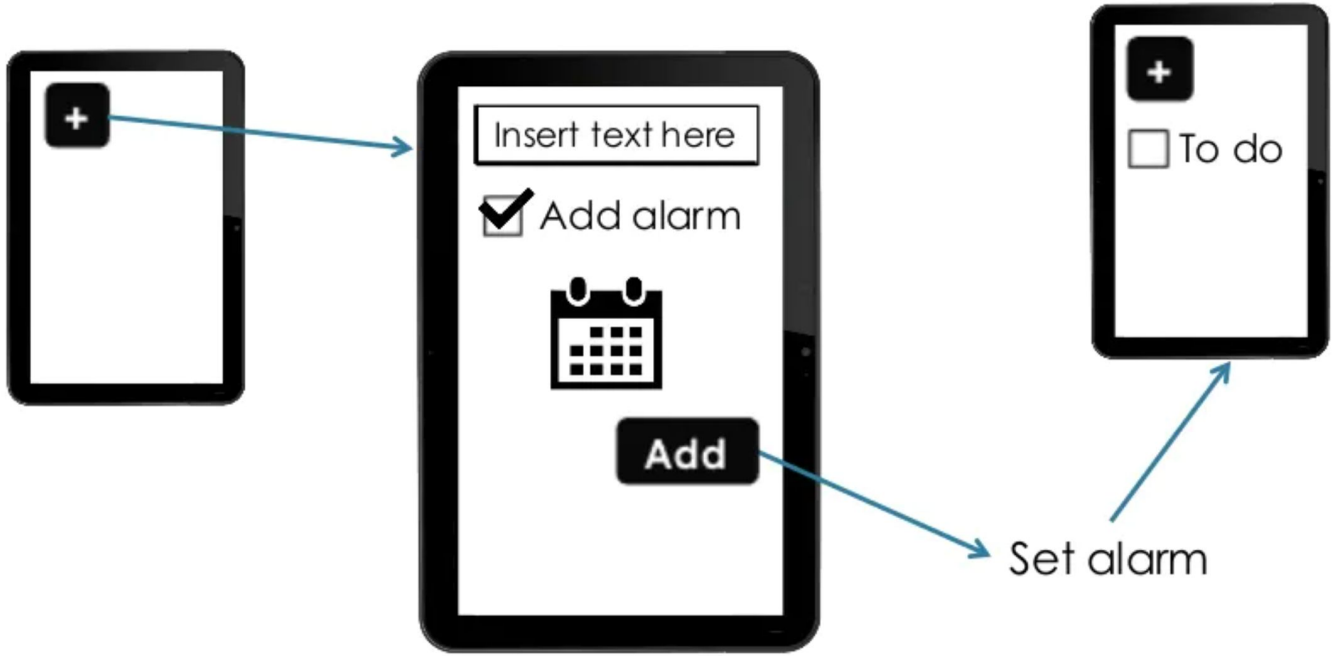
Deadline is reached

A **popup message** appears on the screen

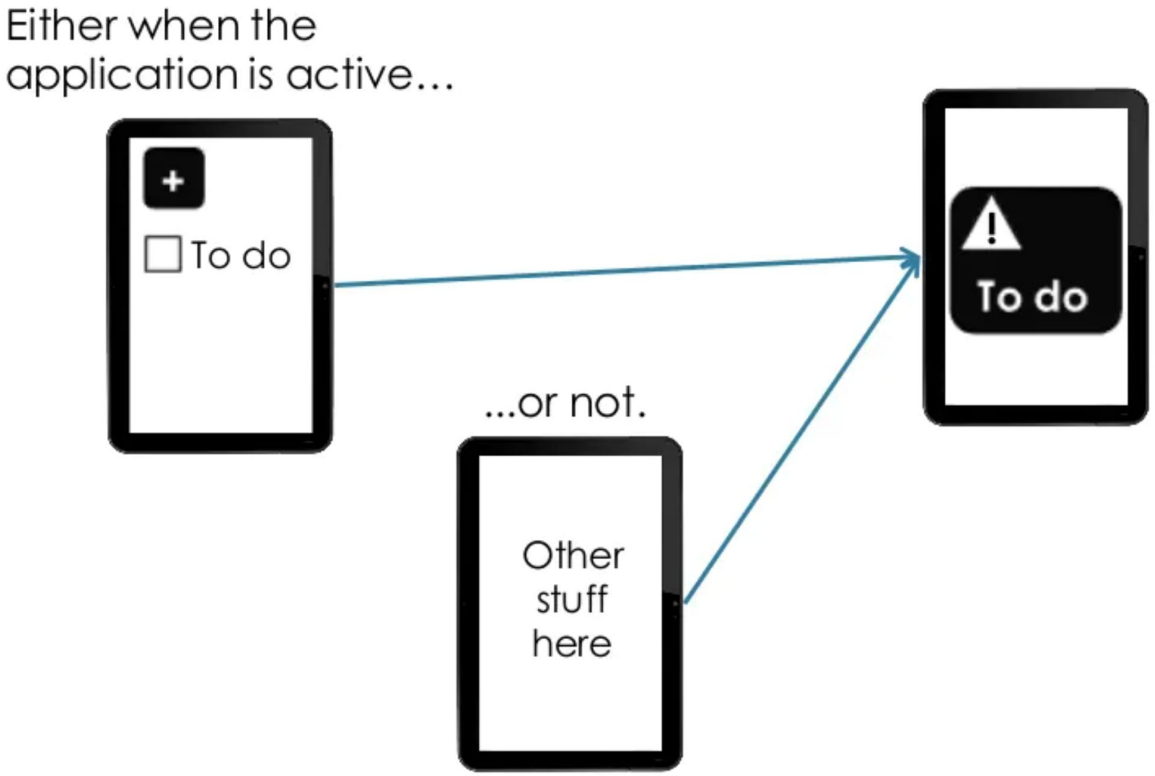
The user is notified: the task has to be completed

CC BY NC

Add both taking note and reminder in same app

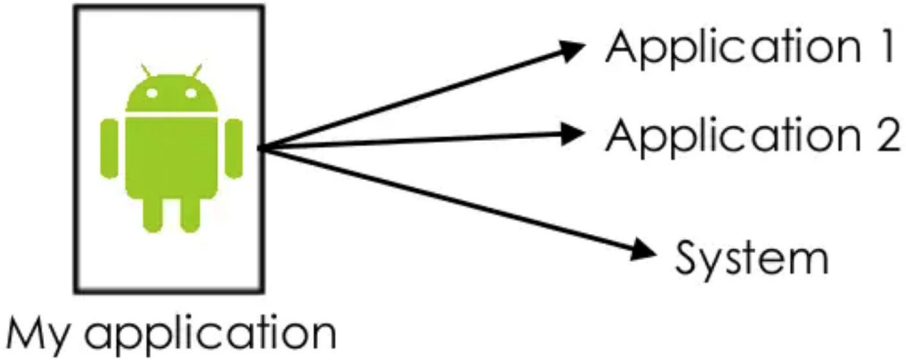


Notify the User



Using Intents to Broadcast Events

Intents are able to send messages **across process boundaries**



You can implement a **Broadcast Receiver** to listen for (and respond to) these broadcast messages

CC BY NC

Broadcast Intents

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Broadcast vs. Activity

Use implicit intents (with ACTION) to send broadcasts or start activities

Sending broadcasts

- Use sendBroadcast()
- Can be received by any application registered for the intent
- Used to notify **all** apps about an event
- Example?

Starting activities

- Use startActivity()
- Find a single activity to accomplish a task
- Accomplish a specific action

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Implicit Intent Declaration

```
Intent intent = new Intent(actionString);
intent.putExtra(extraName, extraValue);
sendBroadcast(intent);
```


CC BY NC

Broadcast Receivers

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

What is a broadcast receiver?

- Listens for incoming intents initiated by system or using sendBroadcast()
  - Everything happens in the background
- Intents is sent
  - By the system, when an event occurs that might change the behavior of an app
    - Changes in network activity (WiFi/Data connections)
    - Incoming Calls (Skype/Messenger/WhatsApp)
  - By another application, including your own

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Broadcast receiver always responds

- Responds even when your app is closed
- Independent from any activity
- When a broadcast intent is received and delivered to onReceive(), it has 5 seconds to execute, and then the receiver is destroyed
    - Execute?: Store something in DB, play ringtone, show notification, …

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

System broadcasts

- Automatically delivered by the system when certain events occur - Examples
  - After the system completes a boot
    - android.intent.action.BOOT_COMPLETED
  - When the wifi state changes
    - android.net.wifi.WIFI_STATE_CHANGED

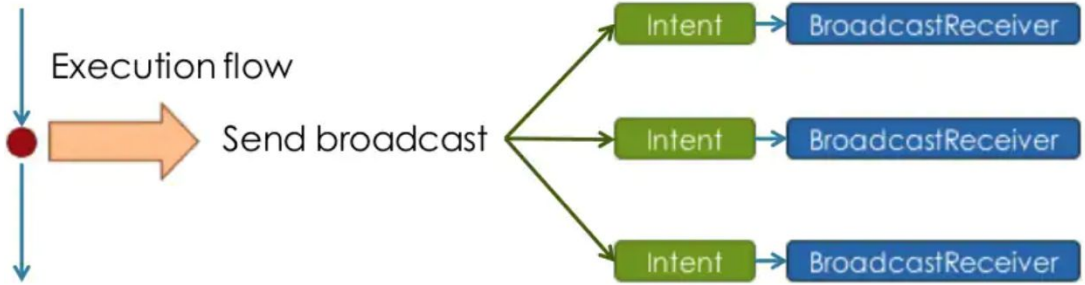
This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Custom Broadcasts

- Deliver any custom intent from the app as a broadcast
  - sendBroadcast() method—asynchronous
  - sendOrderedBroadcast()—synchronously
  - Use custom action name. i.e. android.example.com.CUSTOM_ACTION

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

sendBroadcast()



- All receivers of the broadcast are run in an undefined order
- Can be at the same time
- Efficient
- Use to send custom broadcasts

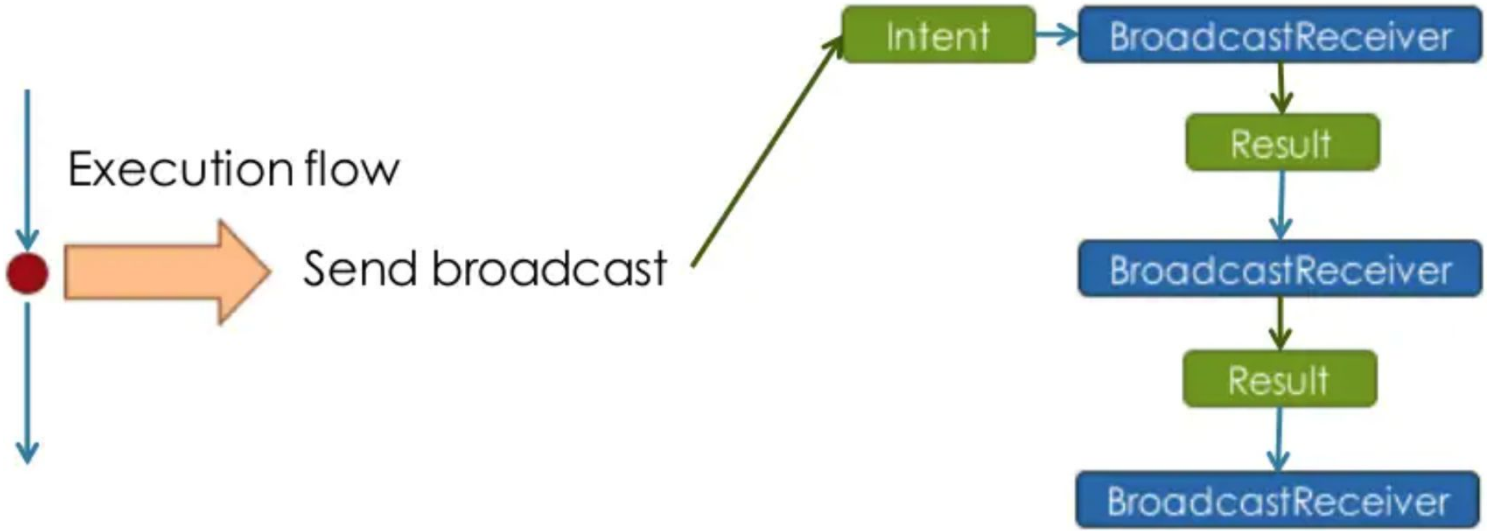
This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

sendOrderedBroadcast()

- Delivered to one receiver at a time
- Receiver can propagate result to the next receiver or abort the broadcast
- Control order with android:priority of matching intent filter
- Receivers with same priority run in arbitrary order

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

sendOrderedBroadcast()



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Implementing Broadcast Receivers

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Steps for creating a broadcast receiver

1. Inherits BroadcastReceiver
2. Implement onReceive() method
3. Register to receive broadcast
    - Statically, in AndroidManifest
    - Dynamically, with registerReceiver()

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

BroadcastReceiver

```
public class CustomReceiver extends BroadcastReceiver {  public
    CustomReceiver() {
    }
    @Override
    public void onReceive(Context context, Intent intent) {
        // TODO: This method is called when the BroadcastReceiver
        // is receiving an Intent broadcast.
        //   Write code to perform some task here.
            // i.e. show notification, start service or …
    }
}
```


This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Register in Android Manifest

- <receiver> element inside </receiver>
- <intent-filter> registers receiver for specific intents

```
<receiver
      android:name=".CustomReceiver"
      android:enabled="true"
      android:exported="true">
            <intent-filter>
       <action android:name="android.intent.action.BOOT_COMPLETED" />
            </intent-filter>
</receiver>
```


This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Available system intents/actions

- ACTION_TIME_TICK
- ACTION_TIME_CHANGED
- ACTION_TIMEZONE_CHANGED
- ACTION_BOOT_COMPLETED
- ACTION_PACKAGE_ADDED
- ACTION_PACKAGE_CHANGED
- ACTION_PACKAGE_REMOVED
- ACTION_PACKAGE_DATA_CLEARED
- ACTION_PACKAGES_SUSPENDED
- ACTION_PACKAGES_UNSUSPENDED
- ACTION_UID_REMOVED
- ACTION_BATTERY_CHANGED
- ACTION_POWER_CONNECTED
- ACTION_POWER_DISCONNECTED

This work is licensed under a Creative Commons Attribution NonCommercial 4.0 International License

Implement onReceive()

```
@Override
public void onReceive(Context context, Intent intent) {  String
    intentAction = intent.getAction();
    switch (intentAction){
        case Intent.ACTION_POWER_CONNECTED:
            break;
        case Intent.ACTION_POWER_DISCONNECTED:
            break;
    }
}
```


This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Custom Broadcasts

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Custom broadcasts

- Sender and receiver must agree on unique name for intent (action name)
- Define in activity and broadcast receiver

```
private static final String ACTION_CUSTOM_BROADCAST =
        "com.example.android.powerreceiver.ACTION_CUSTOM_BROADCAST";
```


This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Send custom broadcasts

```
Intent customBroadcastIntent = new  Intent(ACTION_CUSTOM_BROADCAST);

sendBroadcast(customBroadcastIntent);
```


This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Register dynamically

- In **onStart()**
- Use registerReceiver() and pass in the intent filter
- Must unregister in onStop()

registerReceiver(mReceiver, mIntentFilter) unregisterReceiver(mReceiver)

```
IntentFilter mIntentFilter = new IntentFilter(ACTION_CUSTOM_BROADCAST);
```


This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Destroy!

```
@Override
protected void onStop() {
    super.onStop();
    LocalBroadcastManager.getInstance(this)
            .unregisterReceiver(mReceiver);}
```


This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Local Broadcast Manager

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Local Broadcast Manager

- For broadcasts only in your app
- No security issues since no cross-app communication

LocalBroadcastManager.sendBroadcast()

LocalBroadcastManager.registerReceiver()

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Register local broadcast manager

```
IntentFilter inFilter = new
IntentFilter(ACTION_CUSTOM_BROADCAST);

LocalBroadcastManager.getInstance(this)
    .registerReceiver(mReceiver, inFilter);
```


This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Pending Broadcast Intent

```
long aTime= System.currentTimeMillis() + 60*1000;

// BroadcastReceiver

Intent intent = new Intent(this, AlarmReceiver.class);

Intent.putExtra("alarmTime", aTime);


// call broadcast using pendingIntent

pendingIntent = PendingIntent.getBroadcast(this, 0,
intent, 0);




alarmManager.set(AlarmManager.RTC_WAKEUP, aTime,
pendingIntent);
```

```
public class AlarmReceiver extends BroadcastReceiver {

    private int ATHAN_REQUEST_CODE = 999;
    @Override
    public void onReceive(Context context, Intent intent) {
        long alarmTime = intent.getLongExtra( name: "alarmTime",  defaultValue: -1);

        // write code to show notification and play music

    }
}
```


CC BY NC

Security

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Security

- Receivers cross app boundaries
- Make sure namespace for intent is unique and you own it
- Other apps can send broadcasts to your receiver
  - use permissions to control this
- Other apps can respond to broadcast your app sends
- Access permissions can be enforced by sender or receiver

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Controlling permission sender

- void sendBroadcast (Intent intent,
  String receiverPermission)

- Receivers must request permission with <uses-permission> in AndroidManifest.xml

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Controlling permission receiver

- registerReceiver(BroadcastReceiver, IntentFilter, **String**, android.os.Handler)
- or in <receiver> tag

- Users must request permission with <uses-permission> in AndroidManifest.xml for sending or receiving system broadcast

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License

Apps Business Model

Identifying Which Business Model is Right for Your App

- Start by answering the following four questions:
  - What problem is your app trying to solve and how?
  - What is unique about your app and would people pay for this?
  - What else do you think your app users would be willing to pay for?
  - What business models do your competitors use and how well have they worked?

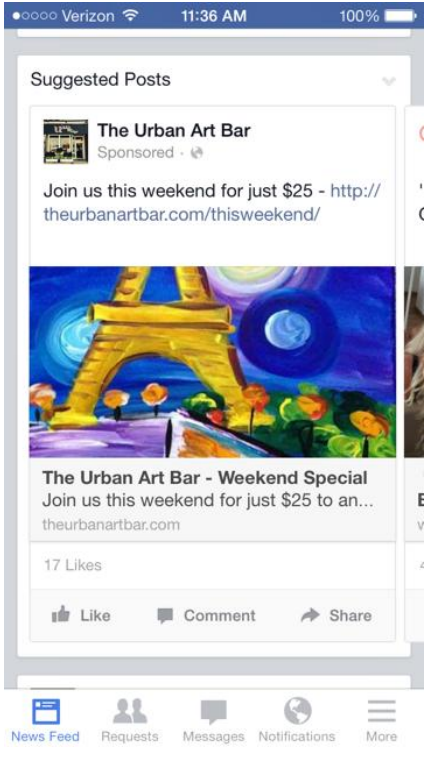
- Find the balance between your need to gain users and earn revenue
  - Some app business models earn more money right off the bat at the expense of quickly acquiring tons of users, while others result in high downloads first and profits later
  - What is your timetable?
    - Can you afford to initially forgo revenue to accumulate users? (It might be worth it, depending on your users)

- App monetization strategies should be chosen and built into app before launch
  - Can always be modified the strategy as time passes or even change it completely
  - But approach mobile with the dual mindset of building an awesome app *and* eventually, a business

Free, but with ads (in-app advertising)

- Removes the cost-barrier to purchasing your app and allow free downloads
- **Goal** – accumulate a sizeable user base and gather information on the people interacting with your app
  - Data is then sorted and sold to ad publishers who pay to place targeted ads in your app

- **Example - Facebook**
  - Users don't directly pay Facebook anything to download or use their mobile platform
  - But leverages a vast amount of their data to sell highly targeted ads, a strategy which has proven to be effective for Facebook
  - The social giant annual ad revenue sat at a staggering
    - $84.169B in 2020 [1]
    - $114. 93B in 2021 [1]



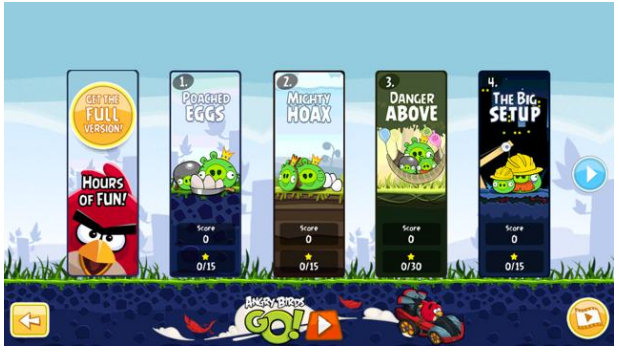
[1] https://www.oberlo.com/statistics/facebook-ad-revenue

Effectiveness: Free, but with ads (in-app advertising)

- **Pros**
  - Mobile apps are in a prime position to collect tons of data on their users
    - Such as their in-app behavior and their location
  - Allows you to gain users quickly because people love free apps
  - Can be effective if moderate and targeted advertising is used
    - Ads are interesting yet limited
- **Cons**
  - Not an innovative model and people find apps annoying, which result in uninstalling
  - Comprise app experience by claiming a portion of the already limited screen size
  - This model won't work for utility apps that are designed to help users perform important functions
    - Ads will be too unnatural and intrusive in this setting when people just want to do something quickly

Freemium (Gated Features)

- **Similar to in-app advertising – offered for free**
  - However, certain features are gated and cost money to be unlocked
  - People have access to an app's basic functionality, but there is a charge for premium or proprietary features
  - The premise of this model is that you attract people to your app and give them a rich preview of what your app can do (without giving them everything)
  - **Goal** – accumulate and engage app users until they are willing to pay for additional in-app tools
- **Example – Angry Birds**
  - The Rovio team (the creator of **Angry Birds**) released a free version of their addictive app
  - However, the app kept certain features hidden (like being able to juice up your bird, additional levels, etc.) until users upgraded (for a small fee) to the full version
  - This allowed people to play Angry Birds and become fans of the game without hesitating at the initial price
  - Once app users have conquered a few levels and have had a glimpse into the game, they're engaged enough to pay for the full version
  - Another example is **Pokemon**

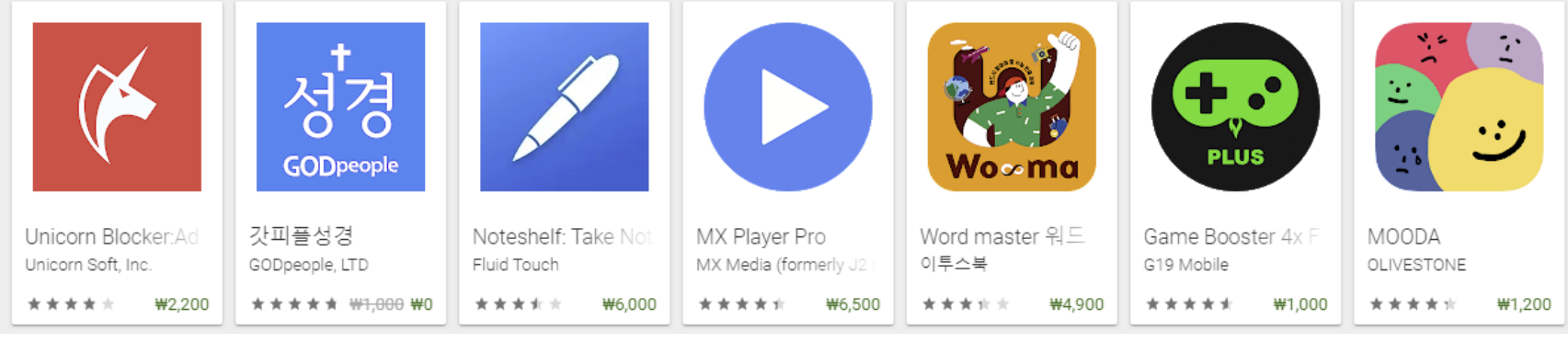


Effectiveness: Freemium (Gated Features)

- **Pros**
  - This mobile app model makes it easy to build up a large user base and showcases your app to people get hooked (unlocking opportunities to upsell)
  - People who “try before they buy” are more likely to become engaged and loyal users further down the line
  - It’s flexible model because it can be adapted to almost any vertical value
- **Cons**
  - If you offer too few features for free, app uninstall will be high
  - If you offer too many features for free, it will be difficult to convince your existing user base to pay for an upgrade (as the upgrade won’t have much incremental value)
  - App marketers must be careful not to provide a large segment of their users (those who are using the free version) with an inferior/bad app experience

Paid Apps (those which you pay to download)

- Simply means your app is not free to download
  - People must first purchase it from an app store
  - Paid apps can cost anywhere between $0.99 and $999.99, and brands generate revenue upfront with each new user
  - The key to success with this model is your ability to showcase the perceived value of your app with a killer app listing (which includes screenshots, five star reviews, etc.) that differentiates your app from your competitors
- **Examples**



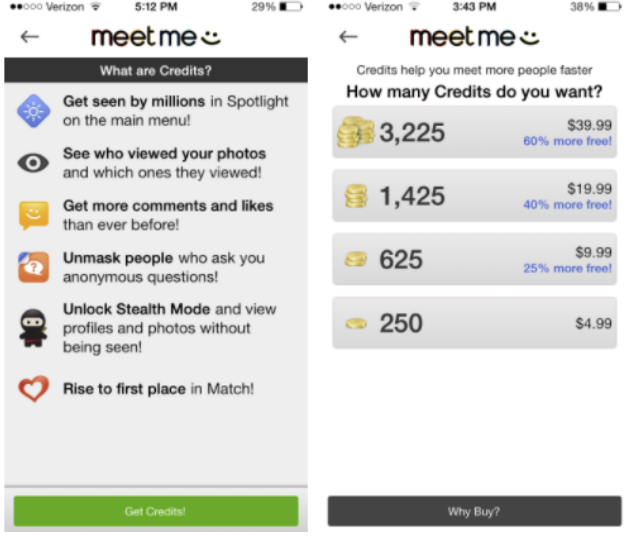
Paid Apps (those which you pay to download)

- **Pros**
  - App developers and app marketers earn revenue upfront with every new download
  - People who have paid for an app are more likely to turn into engaged users (since they spent money to purchase your app vs. choosing a free one)
  - The app does not usually have any in-app advertising thus allowing it to have a cleaner interface
  - This model motivates app developers to focus on innovation
- **Cons**
  - Selling an app is hard because app stores are so overcrowded
    - Stiff competition from many free apps
  - App stores take a cut of the revenue from paid apps
    - Google/Apple gets approximately 30% (recently reduced with condition)
  - Paid models are a shrinking part of app store revenue
  - 90% of paid apps are downloaded less than 500 times per day
    - Cost-barrier to gaining a large number of users

In-App Purchases (Selling Physical/Virtual Goods)

- Selling physical or virtual goods within your app, and then retaining the profits
  - Include a wide variety of consumer goods such as clothes and accessories
  - Can also be virtual goods such as extra lives or in-game currency

- **Example - MeetMe**
  - People can download MeetMe for free and use it to browse profiles, chat with people, and connect with locals
  - However, you can also purchase credits to enhance your visibility and gain new ways to interact with people
  - MeetMe's purchase model is lucrative because the app can clearly highlight the benefits of in-app currency

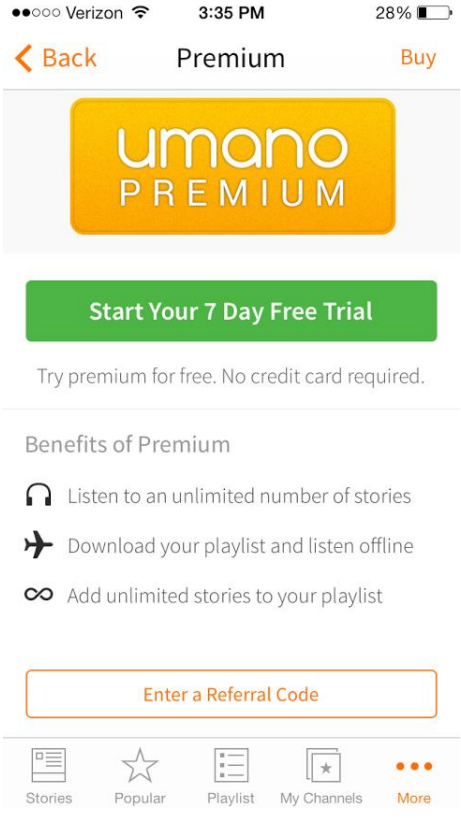


In-App Purchases (Selling Physical/Virtual Goods)

- **Pros**
  - Works particularly well for eCommerce/mCommerce brands and is flexible enough for other verticals too
  - Can help app marketers make comfortable profits with the lowest amount of risk
  - The profit margin is usually high with this model because brands don't have the traditional expenses on mobile that brick-and-mortar stores do (like staffing and rent)
  - Flexible model which can also be adapted to include affiliate programs and partnerships that drive referral revenue
- **Cons**
  - App stores usually take a cut of the revenue for virtual goods (but not physical goods or services) purchased inside an app
  - This model has previously received bad publicity because government officials pressured Apple and Google to add stricter regulations to prevent children from making accidental in-app purchases
  - Apps will need to be more transparent on their app store listing page if they include in-app purchases (which may prevent some people from downloading)

Paywalls (Subscriptions)

- Similar to the freemium model except that it focuses on making available of *contents*, not features
  - Allows an app user to view a predetermined amount of content for free and then prompts them to sign up for a paid subscription to get more
  - Best suited for service focused apps and allows brands to earn revenue on a recurring basis
- **Example – Umano**
  - Transforms news stories into podcasts
  - Allows users to listen to a limited number of stories until they sign up for a premium subscription
  - With this strategy, people can familiarize themselves with Umano's best features
  - But for a fixed amount of time until they are engaged enough to pay for unlimited use and content
  - Another example is **NetFlix**



Paywalls (Subscriptions)

- **Pros**
  - People get to experience all your app's features which increases session lengths and lowers app uninstalling
  - Results in a continual weekly/monthly/yearly (depending on your setup) flow of revenue since subscriptions usually auto-renew
  - Subscribers are more likely to be loyal and engaged app users
  - Motivate app developers and app marketers to ensure they prepare high-quality content that is worth paying for
- **Cons**
  - Does not easily translate to all types of contents
    - Most suited for news, lifestyle, and entertainment apps since they can limit content like articles read or videos watched
  - It can be difficult to determine where and when to place a paywall
    - what is the right limit to place?

Sponsorship (Incentivized Advertising)

- Sponsorship is probably the newest entrant in the mobile world
  - Sponsorship entails partnering with advertisers, who provide your users with rewards for completing certain in-app actions
  - Brands and agencies pay to be part of an incentive system
  - App earns money by taking a share of the revenue from redeemed rewards
  - This way, you can incorporate advertising into your app that enhances your app's ability to engage users

- Example - RunKeeper
  - RunKeeper uses incentivized advertising to motivate its users to track their running activity with their app to unlock exclusive rewards and promotions
  - This strategy lets RunKeeper monetize their app without disrupting the user's app's experience with banner ads.

Sponsorship (Incentivized Advertising)

- **Pros**
  - Innovative app business model which can be adapted for many verticals
  - This advertising strategy is likely to be better received by app users because it is relevant and related to an app's purpose
  - App developers and marketers earn revenue, advertisers get more ad space, and users benefit from free promos
  - This form of advertising can be aligned with your app's conversion funnels
- **Cons**
  - Mobile marketers need to be careful about what actions they incentivize within their app (Apple has been cracking down on incentivizing downloads and social sharing)
  - This app business model has not been as thoroughly tried and tested as others (results and success may vary)

Apps could opt for a hybrid approach

- As the app landscape becomes more sophisticated, we should expect to see a trend towards more hybrid models
  - For instance, you can start with a “free, but with ads” model and then offer users a paid upgrade to an ad-free version, which is a “freemium” approach
- The main takeaway from:
  - Don’t just do what others have done before
  - Adapt and modify each app monetization strategy to make it work for your requirements
- Standard methods may generate revenue with your app
  - But only the creative and courageous app marketers reap the rewards

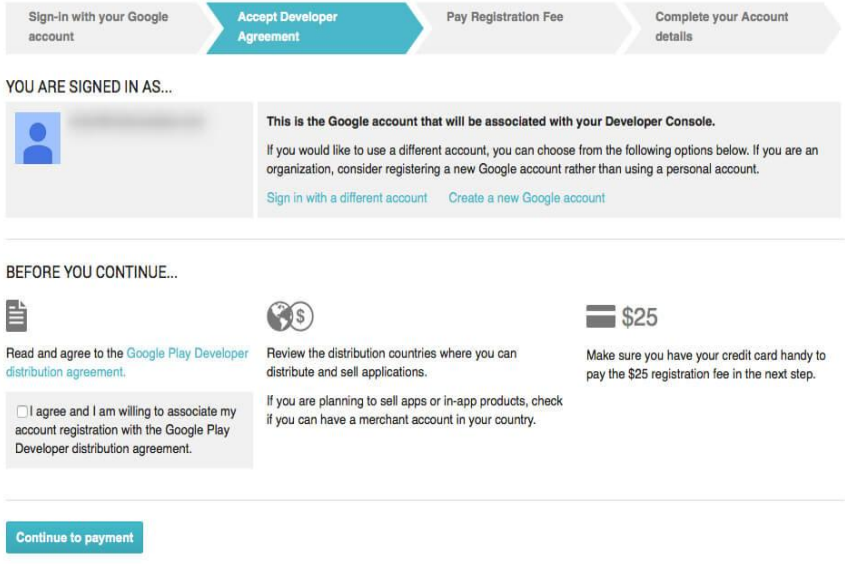
References

https://uplandsoftware.com/localytics/resources/blog/app-monetization-6-bankable-business-models-that-help-mobile-apps-make-money/
https://developer.apple.com/app-store/business-models/
https://bloomidea.com/en/blog/5-business-models-apps
https://www.valuecoders.com/blog/outsourcing-and-off-shoring/types-of-business-models-for-mobile-application-development/

Apps Publishing

Step 1: Create a Google Developer account

- Do at the beginning of the app development process
  - Register a Google Developer Account to publish apps on the Play Market
  - Use any of current Google accounts or create another one to sign up for a Google Developer Account
  - A private or corporate account is fine
  - Any app is transferable to another account
- Creation process
  - Signing the Google Play Developer distribution agreement
  - Adding some personal information
  - Paying a **one-time** registration fee of **$25**
    - Usually, it takes no more than two days to get approval from Google
  - Information can be add or updated later



Step 2: Add a Merchant Account

- If you plan to sell paid apps or in-app purchases, you have to create a Google Merchant Account
  - You may need to provide additional information like a valid photo id (NID/Passport/ResidentCard/DrivingLicence/etc.)
  - Why?
- There you can manage app sales and your monthly payouts, as well as analyze sales reports
- Once you finish creating the Merchant profile, the developer account gets automatically linked to it

- **Note:** You don't need a merchant account for earning from ads
  - For monetizing from ads, you need
    - AdMob account to create and provide ads in your app
    - AdSense account for getting payments and accumulates payments from other platforms (i.e. website, youtube)
  - Payments from ads and sales/in-app purchase are paid by different transactions

Step 3: Prepare the Documents

- Paperwork always requires much effort, especially when it comes to any kind of legal documents
  - It is highly recommended starting to prepare the End User License Agreement (EULA) and Privacy Policy in advance
  - You can take the documents from similar apps as references and create your own based on them, or ask a lawyer to make everything from scratch.
- EULA is an agreement between you as an owner and a user of your product
- It contains:
  - What the users can do with the app, and what they aren't allowed to do
  - Licensing fees
  - Intellectual property information, etc.
  - Terms of Use or Terms and Conditions explain
    - What services you offer the users and how you expect them to behave in return
    - Though Google doesn’t demand Terms of Use, it’s better to publish them
- You can create one document, adding there Privacy Policy and Terms of Use chapters
- Pay special attention to include in the Privacy Policy the following information:
  - A complete list of personal data that is collected, processed and used through the app
  - Technical information that is collected about the device and the installed OS
  - Functional features of the app, its paid and free functionality
  - Place of registration of the company and/or location of the copyright holder of the application
- The chosen legal system and legislation that will be applied in resolving disputes and regulating legal relations
- The terms of subscription
  - Citizenship (residence) of the overwhelming majority of application users
  - Age criteria, the presence of specific content

Step 4: Study Google Developer Policies

- These documents explain how apps need to be developed, updated, and promoted to support the store's high-quality standards
- If Google decides that your product violates some policy chapters, it may be rejected, blocked, or even deleted from the Play Store
- Besides, numerous and repetitive violations may lead to the developer account termination
- **Read also**: Why Google and Apple May Remove Your App and How to Deal With That
- So study all the available information carefully about:
  - Restricted content definition
  - Store listing and promotion
  - Impersonation and intellectual property
  - Rules for monetization and ads
    - **Whatever is displayed in an ad is also consider as the content of your app**
  - Privacy, security and deception regulation
  - Spam and minimum functionality
- Google is constantly working on its policies, and it’s important to monitor the changes and stay up to date even after your app is released

Step 5: Technical Requirements

- You went through the development process, endless testing, and bug fixing, and finally, the “X-day” comes.
- Before moving on to the upload process, you need to check the following things:
  - Unique Bundle ID
    - The package name should be suitable over the life of your application.
    - You cannot change it after the distribution. You can set the package name in the application's manifest file.
- Signed App Release With a Signing Certificate
  - Every application should be digitally signed with a developer's certificate
  - The certificate is used to identify the author of an app and can’t be generated again
- The App Size
  - Google set the limit size of the uploaded file: 100MB for Android 2.3 and higher (API level 9-10, 14 and higher) and 50MB for lower Android versions.
  - If your app exceeds this limit, you can always switch to APK Expansion Files.
- The File Format
  - Two possible release formats are accepted by Google: app bundle and .apk. However, .aab is the preferred one. To use this format, you need to enroll in app signing by Google Play.
- You may learn more about app file technical requirements in the Developer Documents, Prepare for the release guide.

Step 6: Creating the App on the Google Console

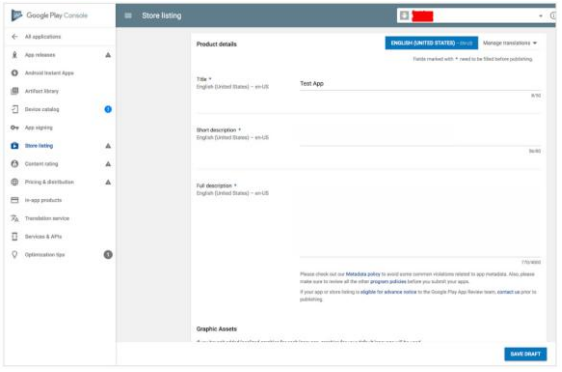
- Now you have the file that is ready for uploading. It’s time to get to the fun part.
- select *Create Application* from the app developer console
- Choose the app’s default language from the drop-down menu
- Add a brief app description (you can change it later)
- Tap on *Create*
- After this, you will be taken to the store entry page, where we will add the complete data about the app.

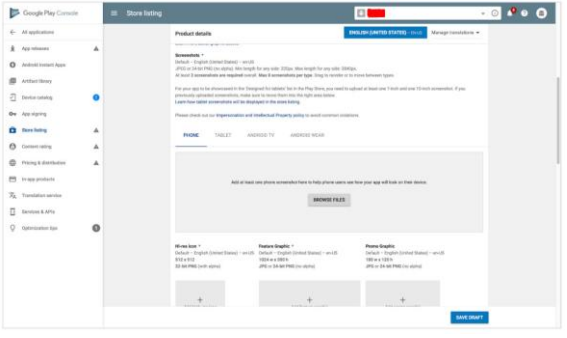
Publish an App

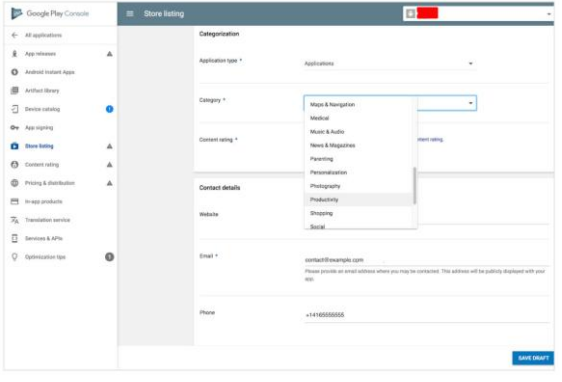
- There are few things that you need to provide with your application while publishing it
- Such as:
  - ✓ Listing details
  - ✓ Store Graphics
  - ✓ Screenshots (Minimum 2/3)
  - ✓ Videos (Optional)

Step 7: Store Listing

- **App Name :-** Give suitable app name for your application.
- **App Description :-** Write suitable description for your app, not exceeding 4000 characters.
- **Category :-** It is required that you select an appropriate main category for you application
  - You can add another four categories as well (used for indexing)
- **Application Type :-** It is also required that you set a category for your application. This could be either set **as application or game**
- **Organization Name :-** It is also required that you mention the name of organization while filling up the details.
- **Support information :-** This includes URL , Email, Phone number etc.







Store Graphics

- **App Icon**
  - ✓ 512 x 512 px
  - ✓ 32-bit PNG
  - ✓ Max size 1024KB
  - ✓ Transparency allowed

- **Feature graphic** This is also optional but is required to get your app featured anywhere. This image has the following constraints
  - ✓ 1024 x 500 px
  - ✓ 24-bit PNG
  - ✓ **No Transparency**

Screenshots

- Screenshots are important because people will see screenshot before installing app on phone. So it must convey your main functionality.

- **Constraints for the screenshots** are
  - ✓ 5/6 additional optional
  - ✓ 480 x 800 px, or 480 x 854 px
  - ✓ 72dpi, RGB, flattened
  - ✓ No transparency
  - ✓ High quality JPEG or 24-bit PNG
  - ✓ No borders
  - ✓ Can show status bar

Video (Optional)

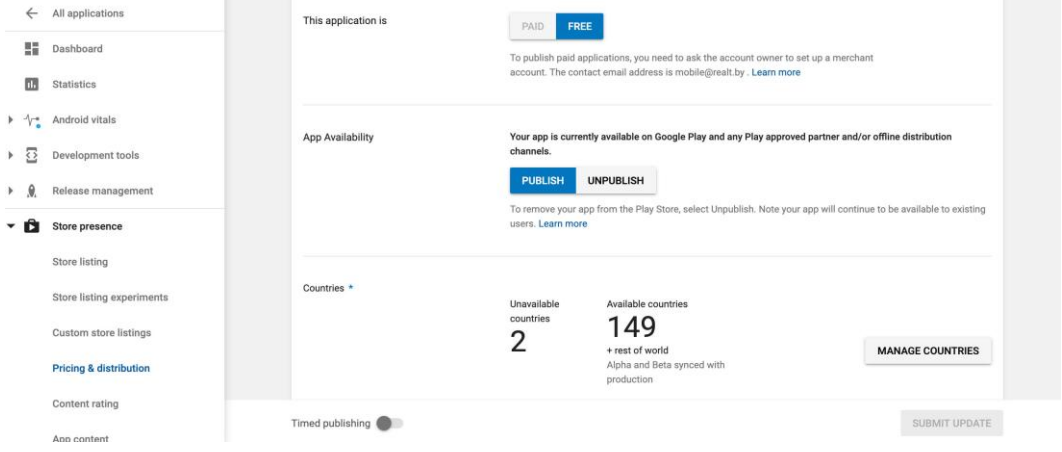
- Finally you can even show how your app works through a video
- This can greatly increase the chances of popularity of your app as the viewer can get synopses of your application
- Google recommends the length of the video to be anything between 30 seconds to 2 minutes

Step 8: Content Rating

- In order not to be marked as an Unrated App (that may lead to app removal), pass a rating questionnaire.
  - You can easily find this section on the left-side menu.
- The information provided in the questionnaire must be accurate.
- Any misrepresentation of your app's content might lead to suspension or removal of the Play Store account.
- Click on *Save Questionnaire* once you complete the survey
- Click on *Calculate Rating*
- In the end, click on *Apply Rating* to confirm the rating and move forward with the pricing & distribution plan

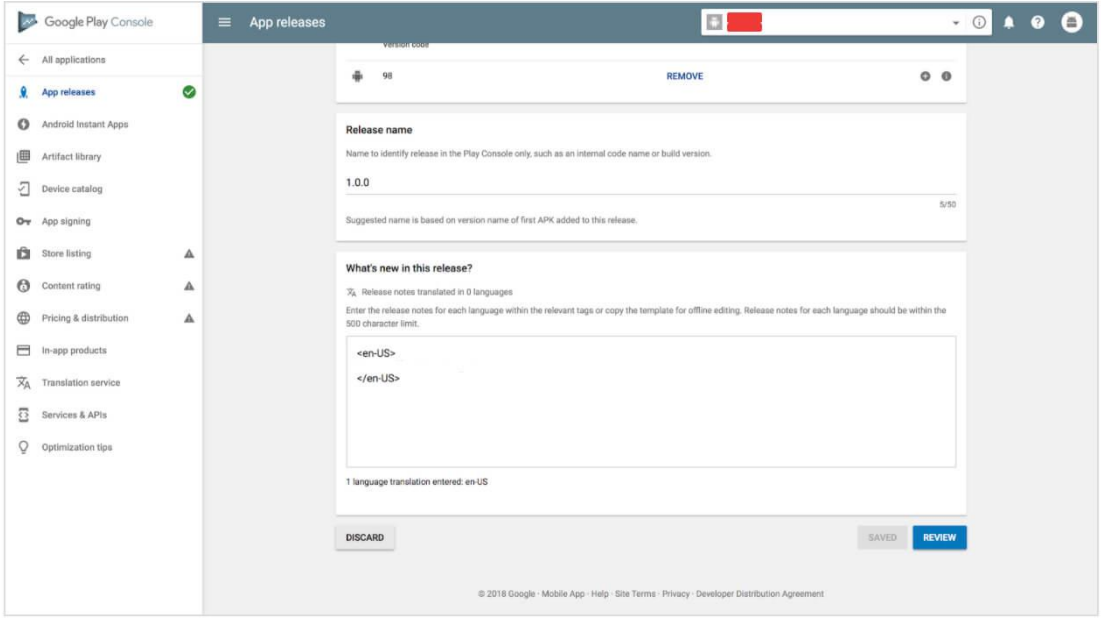
Step 9: Pricing the Application

- In the *Pricing and distribution* section, you need to fill the following information:
  - Whether your app is free or paid
  - Where the app will be available (just choose the countries from the list)
  - Whether your app will be available only on the specific devices
  - Whether the app has sensitive content and is not suitable for children under the age of 13
  - Whether your app contains ads
- Remember that you can change your paid app to a free one later, but you cannot do the vice versa. If you decide later that you want to distribute it for money, you'll have to create another app.



Step 10: Upload APK/AAB and Send for Review

- Finally, you are ready to upload your app file.
- Go to the *App Releases* section on the left panel
  - Three options for publishing the app: Production, Beta and Alpha tracks.
- The Alpha version assumes closed testing and is available only to those who you invite as testers
- The Beta version means that anyone can join your testing program and send feedback to you.
- Recommended starting with Alpha or Beta versions
  - After passing the review process, your app will not be available to everyone on the Play Store.



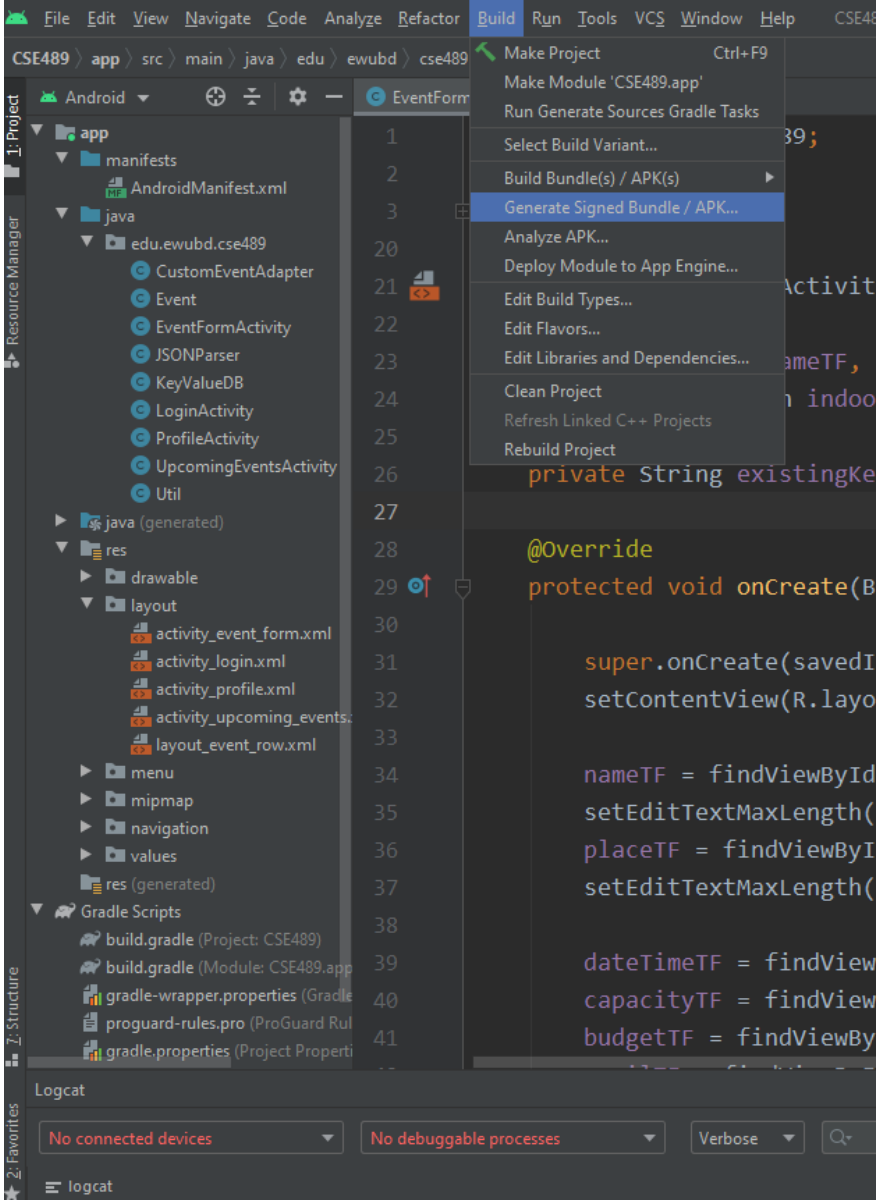
Final Check

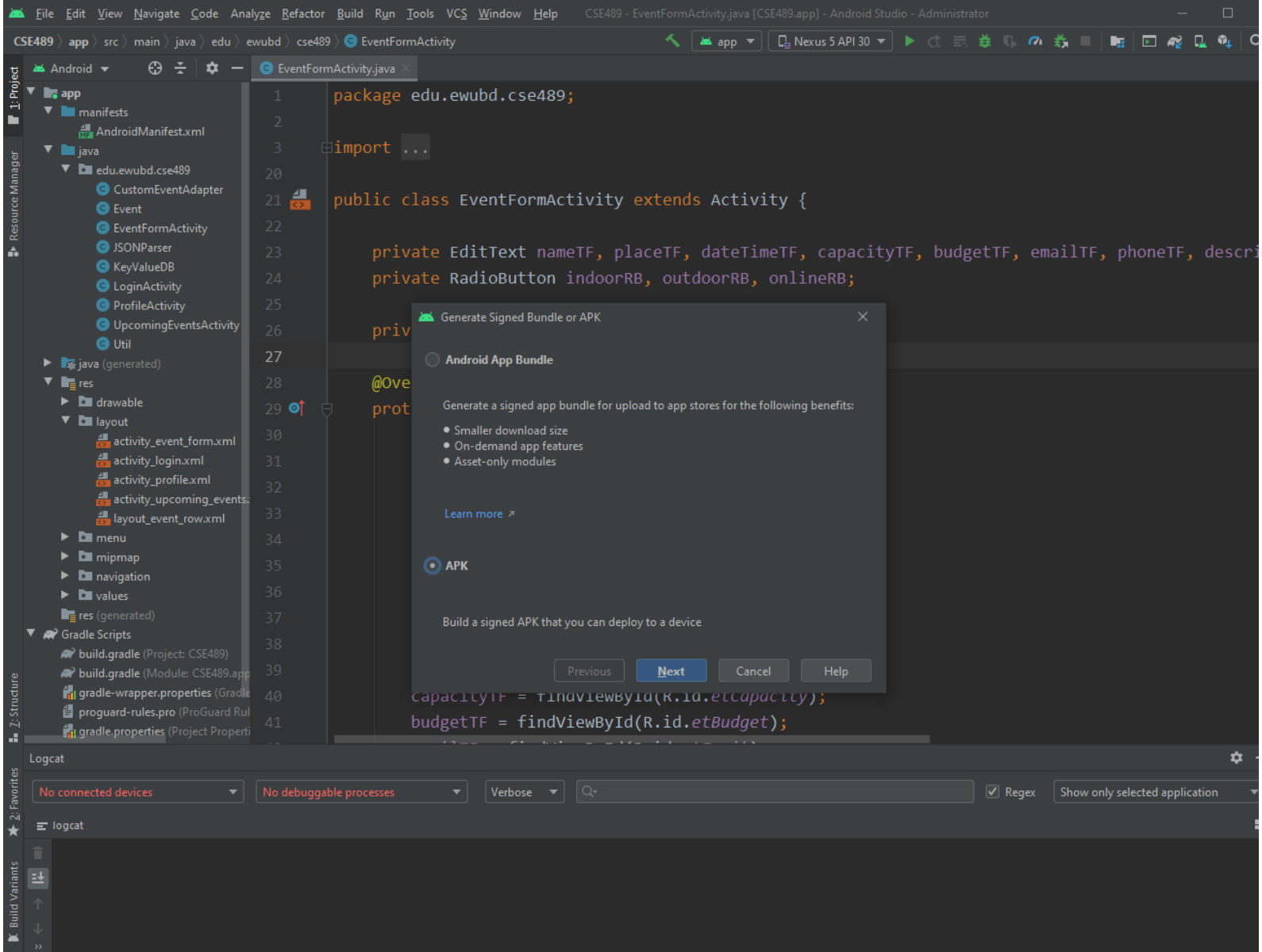
- Apart from this you need to set some other parameters as well. Like

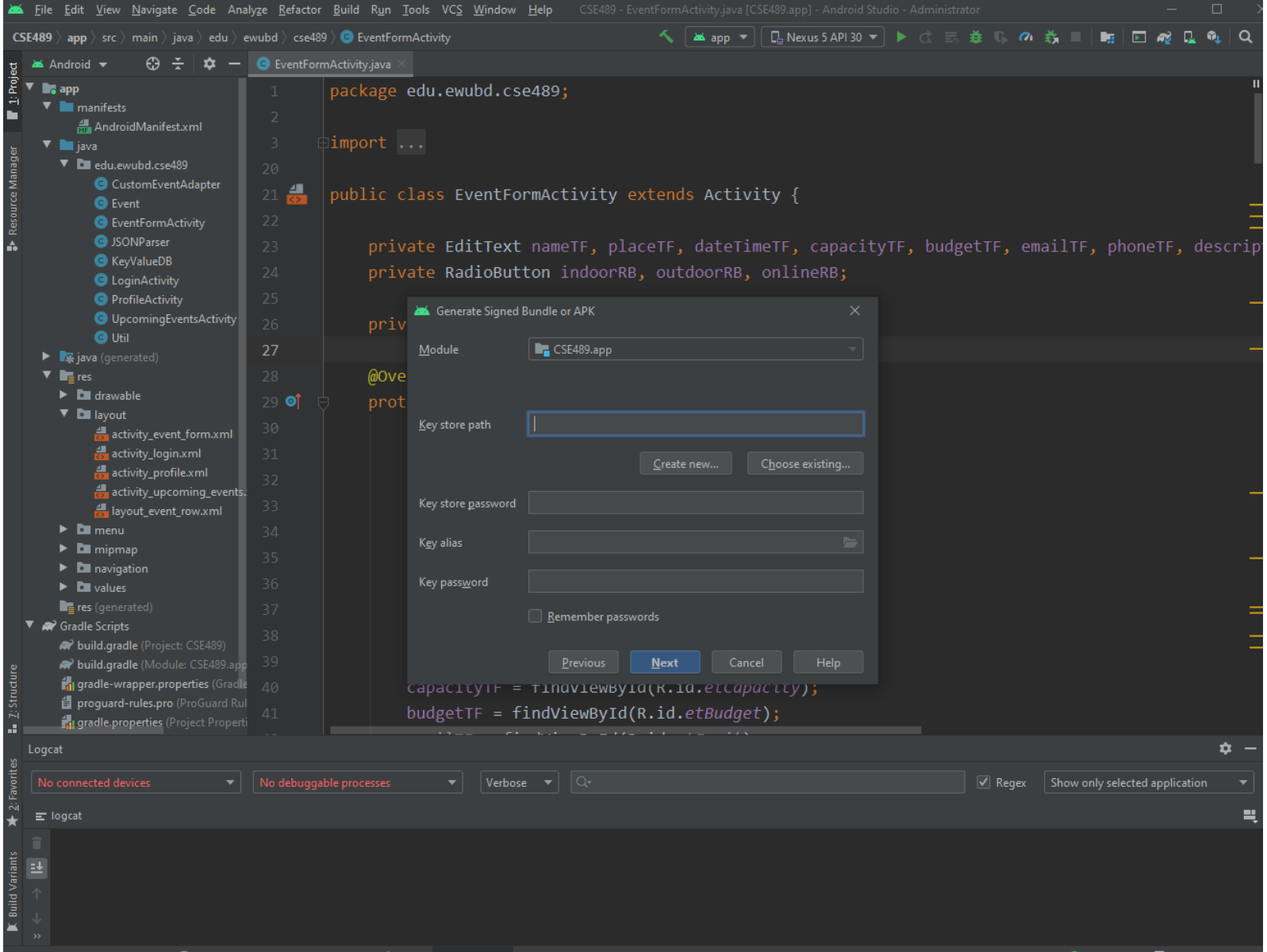
  - ✓ **Size- the size of the app should not exceed 50 Mb**
  - ✓ **Pricing- you need to decide whether your app should be free or priced.**
  - ✓ **Country distribution**- you get to decide which country or territories your app should be distributed.
  - ✓ **Rating-** This is the maturity rating required of your app. You can set is as everyone, low maturity, medium maturity, high maturity.

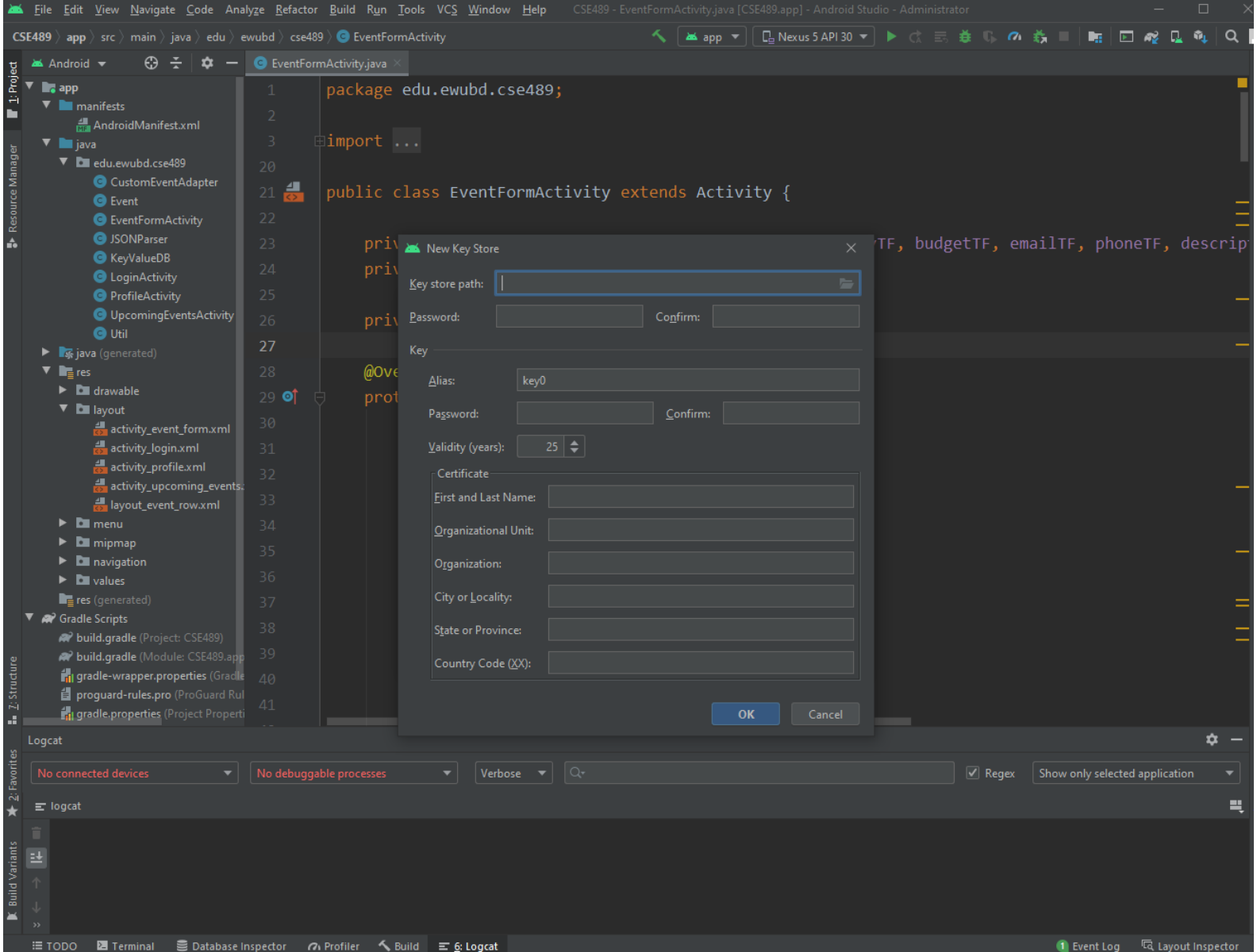
- **At least once you need to read Google play store policies** □
- This APK is then uploaded. This includes
  - ✓ Optimization
  - ✓ Building
  - ✓ signing with your release key
  - ✓ Final testing.
- **All this and you are good to go.**

Exporting APK from Android Studio


File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
CSE489 app src main java edu ewubd cse489
Make Project Ctrl+F9
Make Module 'CSE489.app'
Run Generate Sources Gradle Tasks
Select Build Variant...
Build Bundle(s) / APK(s)
Generate Signed Bundle / APK...
Analyze APK...
Deploy Module to App Engine...
Edit Build Types...
Edit Flavors...
Edit Libraries and Dependencies...
Clean Project
Refresh Linked C++ Projects
Rebuild Project
Android
1: Project
Resource Manager
app
manifests
AndroidManifest.xml
java
edu.ewubd.cse489
CustomEventAdapter
Event
EventFormActivity
JSONParser
KeyValueDB
LoginActivity
ProfileActivity
UpcomingEventsActivity
Util
java (generated)
res
drawable
layout
activity_event_form.xml
activity_login.xml
activity_profile.xml
layout_event_row.xml
menu
mipmap
navigation
values
res (generated)
Gradle Scripts
build.gradle (Project: CSE489)
build.gradle (Module: CSE489.app)
gradle-wrapper.properties
proguard-rules.pro (ProGuard Rul
gradle.properties (Project Properti
7: Structure
2: Favorites
EventForm
private String existingKe
@Override
protected void onCreate(B
super.onCreate(savedI
setContentView(R.layo
nameTF = findViewById
setEditTextMaxLength(
placeTF = findViewByI
setEditTextMaxLength(
dateTimeTF = findView
capacityTF = findView
budgetTF = findViewBy
Logcat
No connected devices
No debuggable processes
Verbose
logcat


File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
CSE489 - EventFormActivity.java [CSE489.app] - Android Studio - Administrator
CSE489 app src main java edu ewubd cse489 EventFormActivity
app
Nexus 5 API 30
Android
EventFormActivity.java
app
manifests
AndroidManifest.xml
java
edu.ewubd.cse489
CustomEventAdapter
Event
EventFormActivity
JSONParser
KeyValueDB
LoginActivity
ProfileActivity
UpcomingEventsActivity
Util
java (generated)
res
drawable
layout
activity_event_form.xml
activity_login.xml
activity_profile.xml
activity_upcoming_events
layout_event_row.xml
menu
mipmap
navigation
values
res (generated)
Gradle Scripts
build.gradle (Project: CSE489)
build.gradle (Module: CSE489.app
gradle-wrapper.properties (Gradle
proguard-rules.pro (ProGuard Rul
gradle.properties (Project Properti
package edu.ewubd.cse489;
import ...
public class EventFormActivity extends Activity {
private EditText nameTF, placeTF, dateTimeTF, capacityTF, budgetTF, emailTF, phoneTF, descri
private RadioButton indoorRB, outdoorRB, onlineRB;
priv
@Ove
prot
Generate Signed Bundle or APK
Android App Bundle
Generate a signed app bundle for upload to app stores for the following benefits:
Smaller download size
On-demand app features
Asset-only modules
Learn more
APK
Build a signed APK that you can deploy to a device
Previous
Next
Cancel
Help
budgetTF = findViewById(R.id.etBudget);
Logcat
No connected devices
No debuggable processes
Verbose
Regex
Show only selected application
logcat


File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
CSE489 - EventFormActivity.java [CSE489.app] - Android Studio - Administrator
CSE489 app src main java edu ewubd cse489 EventFormActivity
app
Nexus 5 API 30
Android
EventFormActivity.java
app
manifests
AndroidManifest.xml
java
edu.ewubd.cse489
CustomEventAdapter
Event
EventFormActivity
JSONParser
KeyValueDB
LoginActivity
ProfileActivity
UpcomingEventsActivity
Util
java (generated)
res
drawable
layout
activity_event_form.xml
activity_login.xml
activity_profile.xml
activity_upcoming_events.
layout_event_row.xml
menu
mipmap
navigation
values
res (generated)
Gradle Scripts
build.gradle (Project: CSE489)
build.gradle (Module: CSE489.app
gradle-wrapper.properties (Gradle
proguard-rules.pro (ProGuard Rul
gradle.properties (Project Properti
1 package edu.ewubd.cse489;
3 import ...
21 public class EventFormActivity extends Activity {
23 private EditText nameTF, placeTF, dateTimeTF, capacityTF, budgetTF, emailTF, phoneTF, descrip
24 private RadioButton indoorRB, outdoorRB, onlineRB;
26 priv
27
28 @Ove
29 prot
40 capacityTF = findViewById(R.id.etCapacity);
41 budgetTF = findViewById(R.id.etBudget);
Generate Signed Bundle or APK
Module
CSE489.app
Key store path
Create new...
Choose existing...
Key store password
Key alias
Key password
Remember passwords
Previous
Next
Cancel
Help
Logcat
No connected devices
No debuggable processes
Verbose
Regex
Show only selected application
logcat
1: Project
Resource Manager
7: Structure
2: Favorites
Build Variants


File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
CSE489 - EventFormActivity.java [CSE489.app] - Android Studio - Administrator
CSE489 app src main java edu ewubd cse489 EventFormActivity
app
Nexus 5 API 30
Android
EventFormActivity.java
app
manifests
AndroidManifest.xml
java
edu.ewubd.cse489
CustomEventAdapter
Event
EventFormActivity
JSONParser
KeyValueDB
LoginActivity
ProfileActivity
UpcomingEventsActivity
Util
java (generated)
res
drawable
layout
activity_event_form.xml
activity_login.xml
activity_profile.xml
activity_upcoming_events.
layout_event_row.xml
menu
mipmap
navigation
values
res (generated)
Gradle Scripts
build.gradle (Project: CSE489)
build.gradle (Module: CSE489.app
gradle-wrapper.properties (Gradle
proguard-rules.pro (ProGuard Rul
gradle.properties (Project Properti
1 package edu.ewubd.cse489;
2
3 import ...
20
21 public class EventFormActivity extends Activity {
22
23 priv
24 priv
25
26 priv
27
28 @Ove
29 prot
30
31
32
33
34
35
36
37
38
39
40
41
TF, budgetTF, emailTF, phoneTF, descrip
New Key Store
Key store path:
Password:
Confirm:
Key
Alias:
key0
Password:
Confirm:
Validity (years):
25
Certificate
First and Last Name:
Organizational Unit:
Organization:
City or Locality:
State or Province:
Country Code (XX):
OK
Cancel
Logcat
No connected devices
No debuggable processes
Verbose
Regex
Show only selected application
logcat
TODO
Terminal
Database Inspector
Profiler
Build
6: Logcat
Event Log
Layout Inspector